需求和架构 (2) [07-Raw/26生成式软件工程/NJU]

> 原视频：https://www.bilibili.com/video/BV1JGap6LE74 · 课程合集：https://space.bilibili.com/202224425/lists/8942017 · 讲者：蒋炎岩（NJU《生成式软件工程》2026）

全课主线

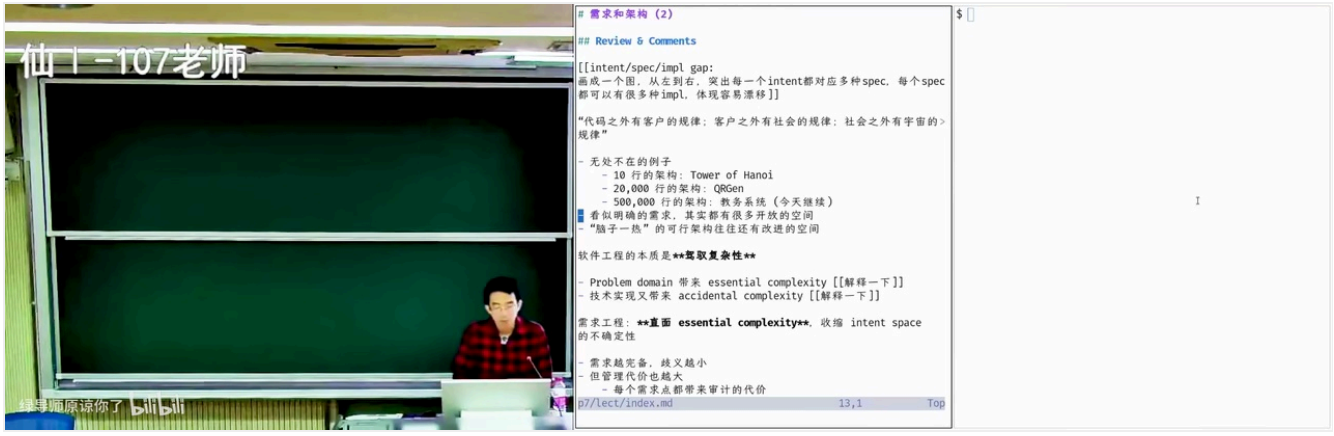
这节课的核心主张是：需求与架构分不开，软件工程的本质是驾驭复杂性；物理世界带来的 essential complexity 消不掉，而人的能力有限，所以必须用架构、抽象、层次化分解和协议来隔离/管理复杂性。讲者想让听众相信，好架构不是把需求清单一次性实现，而是能生长、把扩展作为一等公民并支持组合，从而抵抗需求变更、避免技术债。因此学习时要回到经典与第一手材料，借助 AI 降低理解成本，但核心知识仍要自己建立。

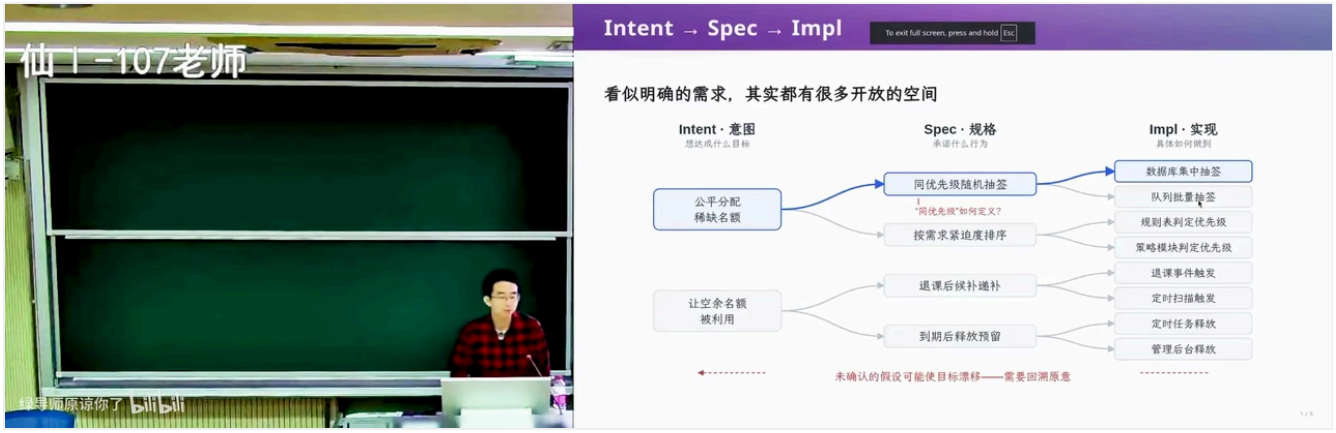
1. 复杂产品先面对 intention、specification、implementation 之间的 gap，需求与架构因此无法分开；看需求还要选层级，因为站在代码、客户、社会、宇宙看到的规律不同。（第1部分）
2. 即使交付 100% 满足客户需求的产品，从第一次设计起就背上技术债，维护中可能陷入长期泥潭；所以软件工程的本质是驾驭复杂性，而物理世界在信息世界中的投影会带来无法消除的 essential complexity。（第1、2部分）
3. 既然 essential complexity 消不掉，架构就不是消灭它而是管理它；数据库或文件系统等实现选择都是 trade-off，属于 accidental complexity，需求工程则直接面对 essential complexity，需要不停澄清客户到底要什么。（第2部分）
4. 需求澄清会不断展开，登录这一小点就能引出无穷细节；因此需求既要用 MVP 深度优先，又要在登录方式、界面布局等决策上广度优先，最终还要以某种方式写出来。（第2、3部分）
5. 代码本身也可承载需求，但让 AI 读代码反推需求会得到很长清单，缺少 top-down 的设计理由；（整理者串联）所以还得理解 why，而不只是重建 what。（第3、4部分）
6. 架构的第一个主要作用是分解 essential complexity：把无法避免的复杂度隔离在模块里，通过接口切分让其余部分不必承担；数据库、统一身份认证和财务系统集成都说明这种隔离与打通的边界。（第4、5部分）

7. 需求收缩 intent space 的不确定性，架构收缩 specification/implementation space 的可能性，但一旦确定就背上技术债；Dijkstra 提醒人的能力有限，因此需要抽象与层次化分解，而结构化程序设计早年已在处理“人不能一次理解复杂系统”。 （第5、6、7部分）
8. 抽象和层次化分解落到机制上，就是私有 memory 加共享空间时用通信协议隔离复杂性，并把数据与操作打包成概念逐层搭建；DP 难调的例子说明代码若丢掉状态空间抽象，就应先把状态与迁移显式化、可观察，再优化。 （第7、8、9部分）
9. AI 与 web search 降低了读经典、从大量论文中找好抽象的成本，但建立 CS 核心知识前仍会痛苦；《Growing a Language》和语言设计史说明语言/架构要能生长，不能只在极简与大而全间二选一，C→C++ 也说明能长不等于长得好，可生长架构要求扩展一等且可组合。 （第6、10、11、12、13部分）
10. 经典系统展示了这种抽象与生长：Unix philosophy 用文本流统一接口，让工具可组合；关系数据库用二维表/schema/事务/视图隔离存储复杂性并支持扩展；Internet 只做 networking/connectivity，以变化本身为永存原则。 （第14、15、16、17、18部分）
11. 在系统层之上，经典业务架构把需求点映射成数据库 transaction 与视图，页面即事务，并用模板语言和 MVC 分工；但 MVVM/React 主要处理 View State，MVC/MVVM 没解决长流程的 essential complexity。 （第19、20、21、22部分）
12. 长流程和数据库把派生结果物化会制造不一致，复式记账用成对、等额、反向记录获得试算平衡；因此 event sourcing 不记当前状态而记状态迁移/不可篡改流水账，用 LLM 编译并重放，使依赖显式、可推演未来，而不是手改数据库。 （第23、24、25、26、27、28部分）

讲者的结论/号召是：承认人的能力有限，把需求与架构连起来，用可生长、扩展一等且可组合的架构去管理复杂性，而不是只按需求清单做一次性实现。

第 1 部分





需求与架构分不开：复杂产品先面对 intention–specification–implementation 的 gap

讲者说，**需求与架构**（转录中作“需求和价格”）有时分不开。从早期课程起，**杰克**就反复讲：当一个软件或产品足够复杂，就会面对 **intention、specification、implementation** 之间的 gap。

- 从最早的**瀑布模型**开始，讲者展示过如何理解这个 gap：用**数据依赖**的方式，做一个 **minimum viable product（MVP，转录中作“minimal variable product”）**，以控制复杂性。
- 讲者给出的逻辑链：产品足够复杂 → 出现 intention/specification/implementation gap → 用数据依赖、MVP 等方式处理这个 gap → 需求与架构无法分开。
- **（整理者串联）** 这里“无法分开”的意思是：需求不是一张孤立清单，它会被架构选择影响，也反过来约束架构选择。

看需求要选层级：代码—客户—社会—宇宙

上节课讲需求和架构时，有一个原则：**代码之外有客户的规律，客户之外有社会的规律，社会之外有宇宙的规律**。讲者强调：你站在哪一个层面去看需求，看到的东西不同。

- 上节课 demo：讲者用**浏览器操作**的方式，逆向教务系统中**与教师相关**的需求，得到一份**需求规格说明书**。讲者认为这份说明书写得很好，符合软件工程实践，也符合做产品、做需求的方式。
- 但作为整个系统的设计者或架构师，要站在需求之外想：当然可以选一个架构、选一个数据库，把这些需求都实现，交付一个**100% 满足客户需求**的产品。

- 讲者随即指出：这样的产品很可能在未来维护中陷入**长期泥潭**。

从第一次设计就背上技术债：100% 满足需求 ≠ 可长期维护

讲者的推理：

1. 你在做任何设计的那一刻起，就背上了一定程度的**技术债**。
2. 这个技术债会一直背下去。
3. 所以即使交付了 100% 满足客户需求的产品，也可能在维护中陷入长期泥潭。

- 讲者还强调：如果编程时注意细节，会发现从**十行代码**、最小的一份代码，到 **50 万行**、**100 万行**、**1000 万行代码**的大系统，**架构无处不在**。
- 上节课的图：最初什么也没有时，就需要做决定；一旦做决定，就会影响未来走向。因此软件架构设计通常有很大的改进空间。
- 对实验的建议：讲者说这两天会把下一个实验放上来。做第一个实验时，可以像他上课那样，直接把需求复制粘贴进去，让 AI 帮着做，然后不停跟 AI 聊天修改。但如果第一反应是“这个东西可以这样做吗？可以这样设计数据库吗？”，第一反应应该是：**也许我可以回过头来再多想一想，可能会有更好的方案**。

Brooks：软件工程的本质是驾驭复杂性

- 讲者引用 Brooks：我们软件工程的本质就是**驾驭复杂性**。
- 当我们在谈需求、看到**需求规格说明书**时，就知道现实中的软件是非常复杂的。

小结：讲者把需求规格引向架构选择与技术债，最后归到 Brooks 的“驾驭复杂性”。

第 2 部分



软件工程的本质是驾驭复杂性

- **讲者主张**：软件工程的本质就是**驾驭复杂性**。
- 看**需求规格说明书**时就能知道：现实中的软件非常复杂。
- 有些复杂性无法消除，属于 **problem domain（原文 ASR: problem dwin）的 essential complexity**。
- 选课问题本身就是这种复杂：
  - 要考虑课程容量、先修要求、时间冲突；
  - 还要排课，还要考虑教室情况；
  - 系统还支持老师和同学都可以借教室。
- 讲者给出的链条：**物理世界就是复杂的 → 物理世界在信息世界里的投影，其复杂性也无法消除 → 消不掉时，用软件架构来更好地管理这种复杂性。**
- 因此，**软件架构**的作用不是消灭 essential complexity，而是更好地管理它。

技术债：数据库 vs 文件系统，都是 trade-off

- 讲者把技术债和实现决定连起来：做任何实现决定，比如用**数据库系统**还是用**文件系统**管理，都会引入一个 **trade-off space**。
- 文件系统一侧：
  - 好处：管理起来很方便，想 hack 它也容易。
  - 代价：性能可能远远不如数据库；有并发控制问题；文件系统本身不支持原子性事务，即 **all-or-nothing**（ASR 作“oh nothing”）。

    - 讲者解释 atomic：可以读多个数据、写多个数据，并让整个读写看起来是 **atomic**，就像上了一把锁、**stop the world** 一样；文件系统没有这个保证。
    - 也有比较疯狂的人想在操作系统里面实现文件系统级的 **transaction**。
- 数据库一侧：
    - 好处：数据库能帮你实现 **transaction**。
    - 代价：每次调试都需要经过一个**中间层**；看数据库时不能像文件系统一样使用所有 UNIX 工具去组合它。
    - 理论上也可以把 query 和其他工具组合，比如让 **awk**（ASR 作“A阵”）做，但那样会引入一个额外的转换层。
- 讲者结论：这些是任何技术选型都会有的 **accidental complexity**，必须和 **essential complexity** 分开。
- 学习各种好的软件架构，目的就是看到：什么时候应该用什么样的架构来减少 **accidental complexity**，以及一个好的系统如何应对世界上的 **essential complexity**。

需求工程直接面对 essential complexity

- 讲者认为，一定程度上，**需求工程**就是去直接面对系统中的 **essential complexity**。
- 它承认软件系统就是复杂的，然后需要不停地和客户沟通，把“我到底要什么”这件事情想清楚。
- 讲者提到两类需求澄清方式：
    1. 经典方式：围绕**需求规格说明书**，比如让 GPT 写一份需求规格说明书。
    2. 另一类方式：使用带 **“grill me”** 的 skill（原文提到 “map cococ”/“GRIMl”，疑似 ASR 误识别；若指具体工具名需核对）。
- 讲者推荐大家试用：做任何一个小的东西，比如做一个**购物车**。
- “grill me”会把所有细节都问一遍：**authentication** 到底怎么做？是接入第三方微信公众号认证，还是自己做一个 **login page**？

第 3 部分



Authentication 怎么做本身就是需求分叉：第三方认证 vs 自建 login page

- 讲者先追问：你到底要怎么做 authentication?
- 选择一：接入第三方认证，例如**微信公众号认证**。
- 选择二：自己做一个 **login page**。
- 讲者强调：这仍然是**需求层面**的东西；每一个选择都会有很大的影响。

把“我要 login”直接交给 GPT，会被猜出一串隐藏需求

逻辑链：

1. 需求只写“我要 login” → GPT 会替你猜一个方案。
2. GPT 可能猜：你需要**注册用户**。
3. 注册用户 → 用户要输入**用户名和密码**；第一次注册时，密码还需要一个 **confirmation**。
4. 继续追问 → 你要不要做**忘记密码**功能?
5. 如果不做忘记密码 → 讲者认为用户一定会忘掉密码；对忘掉密码的用户来说，这是非常糟糕的体验，因为数据还留在里面，但用户没有办法再访问/使用。
6. 如果要做忘记密码 → 要么给他发**短信**，要么给他发**邮件**。
7. 发邮件 → 你用什么来发邮件？问题继续往后展开。
8. 继续往下想 → 你会发现登录这一件事后面有**无穷细节**，进去以后还有各种各样的其他细节。

因为“别人都有用户名密码”而照做，可能已经走进泥潭

- 讲者描述一种常见心理：当你不断往后想，你会觉得 OK，因为所有人都有一个用户名密码登录，所以我的软件也理所应当做一个用户名密码登录。
- 但讲者判断：你可能在做这个设计、这个需求的时候，就已经进入了一个**泥潭**。
- 于是另一个想法出现：那我还是用**微信登录**或者其他第三方登录比较好。

第三方登录看似方便，但有成本与主体门槛

- 讲者观察到：现在注册一个网站时，很多都支持**微信或者支付宝登录**，因为这是一个更方便的模式。
- 但代价是：你要**交钱**；而且对个人开发者可能不是很友好。
- 讲者进一步说：你可能需要一个**公司实体**才能注册。
- 他的理由是：个人承担责任的能力和公司不一样；公司某种程度上需要遵守一定的东西才能注册。
- 结果：这变得非常非常复杂。仅仅登录这一件事，就已经有无穷细节。

需求既要 MVP 深度优先，也要决策广度优先

- 讲者说，深度优先和广度优先其实并不矛盾。
- **深度优先**：先做一个 MVP，快速让你知道最重要的需求/功能点到底有没有被满足、满足到什么程度；也能判断这个软件的需求要不要第一轮、第二轮重做。
- **广度优先**：你的思维方式同样需要一定的广度优先。因为有很多决策，例如登录框到底选哪一种方式来实现、界面到底怎么布局、第一个首页到底怎么展现。
- 这些都是**需求 space** 里的东西。

需求最终要写出来；代码本身也可以承载需求

- 讲者说，直到某一天，你还是需要把需求以某种方式写出来。
- 理论上，**代码本身就是需求**：你可以把一个实际项目的所有文档都删掉，只让大语言模型去运行代码、读代码，它也可以把这些需求点还原出来。

小结：登录这个看似小的点，本身就展开成一条需求链；需求既要用 MVP 深挖，也要用广度优先扫决策，而代码也可以反过来承载需求。

第 4 部分



靠 AI 读代码能反推需求，但结果会是很长的需求点清单，且缺少 top-down 设计理由

- 讲者举例：可以把所有文档删掉，只给一个实际项目，让 AI 运行代码、读代码；它能把需求点还原出来。
- 但这样还原出的需求点会非常非常长。
- 讲者指出/疑问：它没有一个从顶到下说明“为什么这样设计”的理由。
- **（整理者串联）** 代码反推更像在重建“what”，设计理由层面的“why”仍需要额外解释。

设计哲学不是统一答案：简化复杂性、把麻烦交给第三方，但也可以反过来按产品目标权衡

- 讲者认为，整个软件系统设计/需求的宗旨可以是：不要搞太复杂，要简化复杂性，管理好复杂性。
- 链条：如果某个东西实现起来麻烦 → 最好交给第三方来管理 → 自己的系统更简单。这是一种设计哲学。
- 但这不必然适用于所有产品：你的产品可能就需要用户名和密码，需要管理用户数据；讲者举例说甚至可能想偷用户密码、构建自己的密码库。讲者明确说这件事肯定不合规，但在体量足够小的时候，可能有人会想做。
- 所以这是 **trade-off（权衡）**：可能需要在 **深度优先** 和 **广度优先** 之间达到比较好的平衡。

- 结论：只能动手做。你一方面要想一个软件系统为什么做这个设计；如果换另一个设计，它会变好还是变坏。讲者说“我不知道”。
- 讲者问：上节课看完教务系统后，有没有想象如果你来实现这个教务系统，你会怎么做。

需求工程没有特别办法：只能看得多、做得多

- 讲者认为，这些需求工程问题好像没有特别的办法，只能看得多、做得多。
- 他今天也会讲一些具体系统的实现。

架构的第一个主要作用：分解 essential complexity

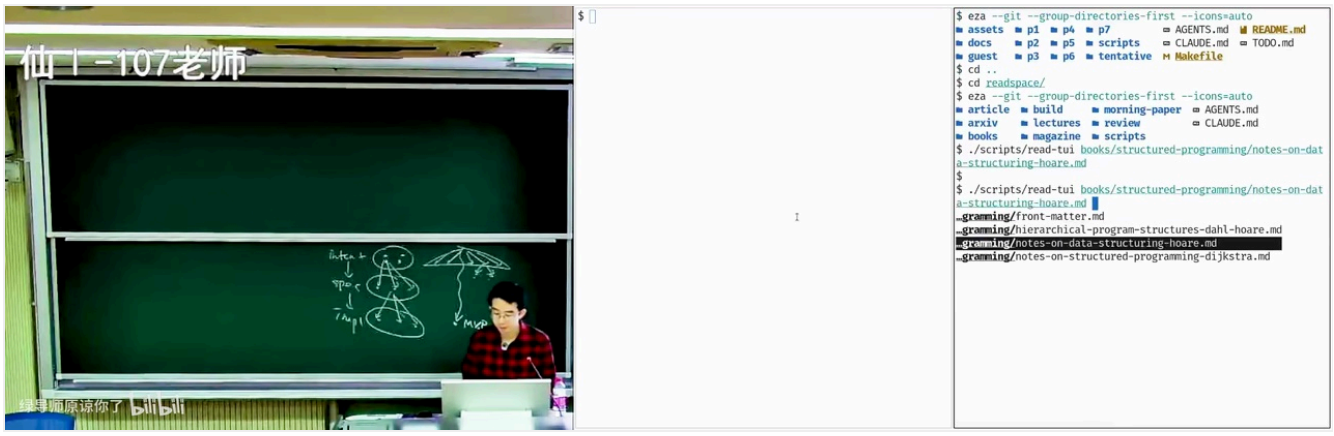
- 需求之外是架构。架构有两个主要作用；第一个是分解 **essential complexity（本质复杂度）**。
- 链条：有些复杂度确实无法避免 → 把它们隔离在不同模块里 → 在模块之间做很好的接口切分 → 让系统其余部分不必承担这些复杂度。
- 例子：**数据库**就是非常好的架构设计。把数据放到数据库后，就不用再关心数据怎么存储；它给你一个很好的 **contract（契约）**。
- 尤其是用云厂商数据库时，一般有备份承诺：数据不会丢；最多一段时间出问题，可能降低 **availability（可用性）**，但备份很快顶上；数据库访问起来几乎没有任何感觉。
- 这样，数据可靠性和数据访问的 **原子性（atomicity）** 都不用担心，可以更好关注业务需求。
- 数据库本身是通用 **system（系统）**，可以把一些逻辑下放给数据库。
- 讲者今天也会讲 MVC。
- **（整理者串联）** 这对应前面“实现麻烦的东西交给第三方管理”的设计哲学。

同一模块化思路也用在自家软件系统：以学校信息系统为例

- 在自己的软件系统里，也可以设计不同模块。
- 学校最希望打造一个全打通的信息系统：填任何信息，不需要在这里填一次、那里填一次。
- 学校花了很长时间才完成**统一身份认证和打通**。

第 5 部分





统一身份认证解决“重复填信息”，但校内仍有系统没打通

- 讲者回顾：目标是不要“这里填一次、那里填一次”，因此花了很长时间完成**统一身份认证和打通**。
- 但学校还有很多系统，听众可能没用过。例如**财务系统**：学生不从学校领工资、不管项目，但研究生可能给老师报销单据，过程很麻烦。
- 财务系统没有完全和现在的系统打通。讲者认为可以理解，因为财务系统有大量**领域知识**：要把账做平、接受审计，这些功能本身就在财务领域里。
- 现状：财务系统和其他部分尚未很好连起来，例如**数据共享**、**通知推送**。讲者希望这是需求和架构能够做到的功能。
- （整理者串联）身份认证打通解决的是入口和重复填写问题；财务系统例子说明，集成还涉及领域数据和领域流程。

需求与架构分别收缩不同空间，但确定后会带来技术债

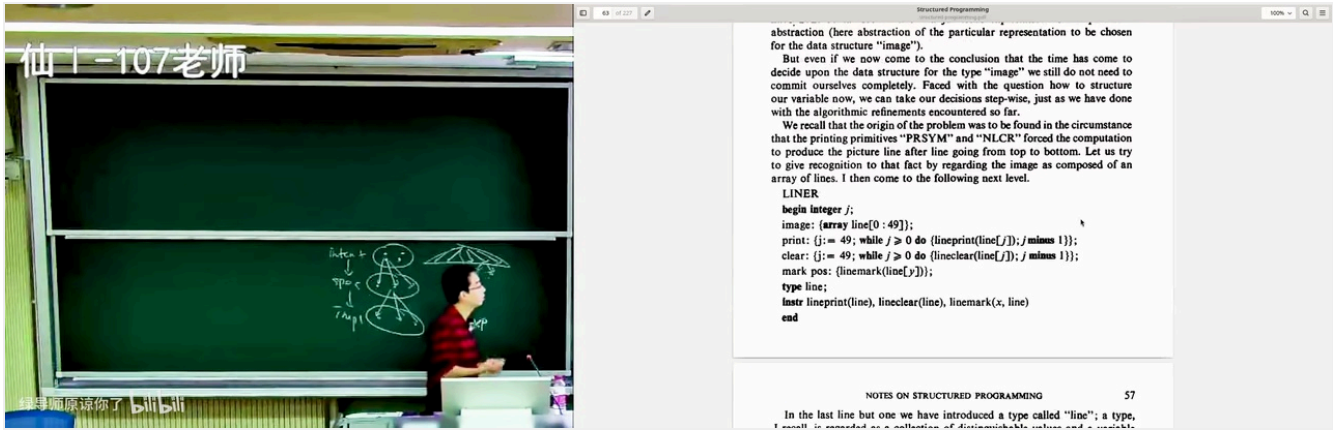
- 讲者把过程表示为：($\text{intent} \rightarrow \text{specification} \rightarrow \text{implementation}$)。
- 一个 intent 可以有多种 specification；一个 specification 又可以有多种 implementation，所以会产生**漂移**。
- **需求**：收缩 **intent space** 的不确定性。
- **架构**：收缩 **specification space** 和 **implementation space** 的可能性。
- 但一旦确定，就会背上一定程度的**技术债**。讲者说明这是上一节课的内容。

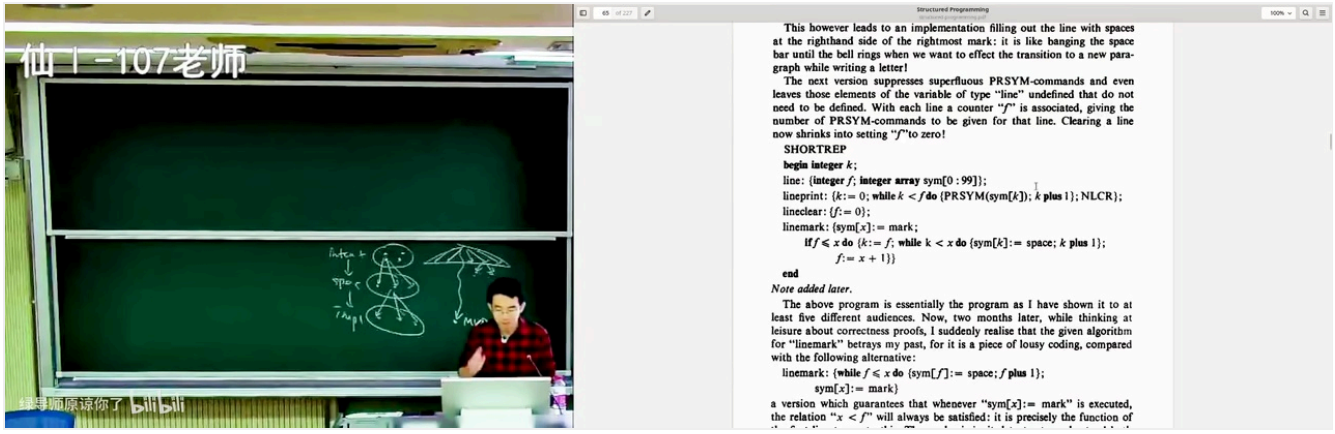
早期软件工程已经在争论程序应怎么构造

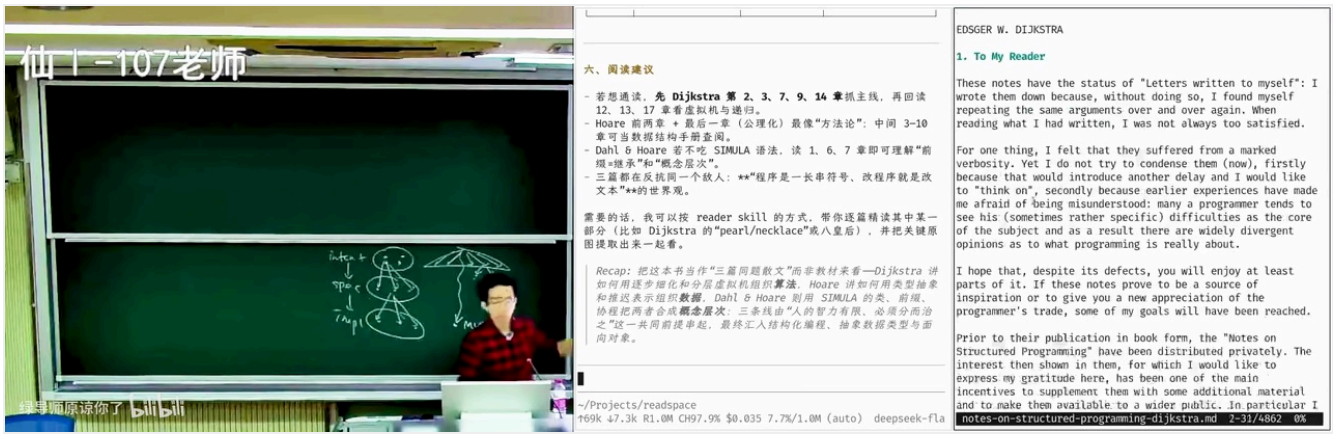
- 讲者自嘲前面内容可能让人困惑，于是提到自己最近重新读了一些老 paper。
- 他发现：在软件工程诞生初期甚至更早，已经有很多大家思考过软件/程序应如何构造。
- 例子是 **Dijkstra 的《Notes on Structured Programming》**（结构化程序设计；原文作“may note some data structure”）。讲者用 PDF 打开这本书。
- 书里的程序看起来不是现代意义上的软件工程书：讲的是相对较小的程序。
- 里面能看到流程图，讲**顺序结构、选择结构、循环结构**。
- 当时的 argument 是：其实不需要 **go to**；只用顺序、选择、循环这种 diagram 方式，就可以表示任何程序逻辑。那时还在争论 **“Go to is harmful”**，甚至是在比较小的函数层面争论。
- 讲者还找了书中的示例程序：看起来非常简单，但已经开始出现结构。讲者认为这本书写得很好。
- 最后讲者说，当然不应该用这种传统方式来读它（具体读法此处未展开）。

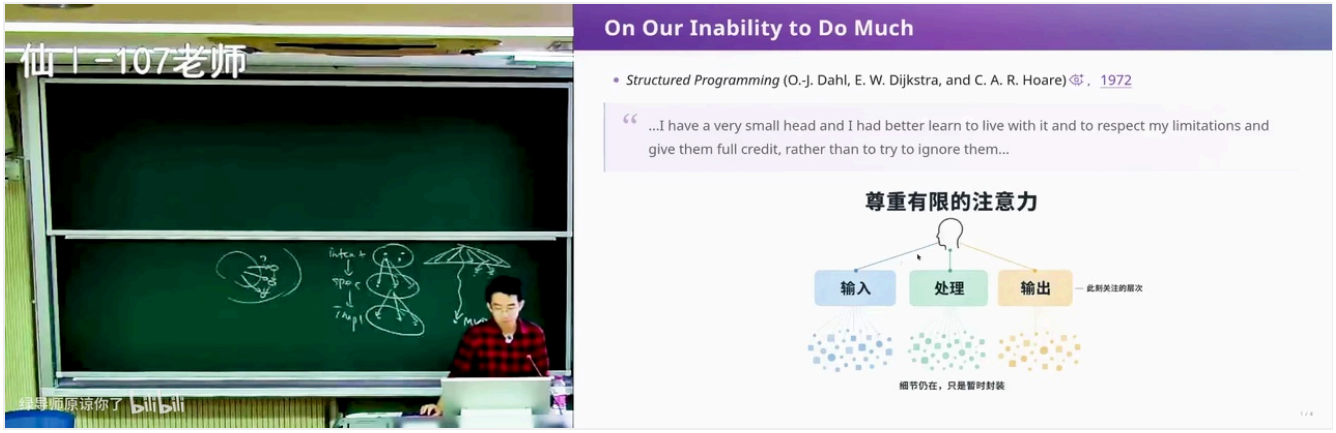
小结：统一身份认证只打通部分系统；架构通过收窄 intent→spec→impl 的漂移来约束实现，但确定选择会带来技术债；讲者由此回到结构化程序设计的历史争论。

第 6 部分


仙丨=107老师
绿哥师原谅你了 bilibili
Structured Programming
abstraction (here abstraction of the particular representation to be chosen for the data structure "image").
But even if we now come to the conclusion that the time has come to decide upon the data structure for the type "image" we still do not need to commit ourselves completely. Faced with the question how to structure our variable now, we can take our decisions step-wise, just as we have done with the algorithmic refinements encountered so far.
We recall that the origin of the problem was to be found in the circumstance that the printing primitives "PRSYM" and "NLCR" forced the computation to produce the picture line after line going from top to bottom. Let us try to give recognition to that fact by regarding the image as composed of an array of lines. I then come to the following next level.
LINER
begin integer j;
image: (array line[0 : 49]);
print: (j:= 49; while j ⩾ 0 do (lineprint(line[j]); j minus 1));
clear: (j:= 49; while j ⩾ 0 do (lineclear(line[j]); j minus 1));
mark pos: (linemark(line[y]));
type line;
instr lineprint(line), lineclear(line), linemark(x, line)
end
NOTES ON STRUCTURED PROGRAMMING 57
In the last line but one we have introduced a type called "line"; a type,


仙丨=107老师
绿哥师原谅你了 bilibili
Structured Programming
This however leads to an implementation filling out the line with spaces at the righthand side of the rightmost mark: it is like banging the space bar until the bell rings when we want to effect the transition to a new paragraph while writing a letter!
The next version suppresses superfluous PRSYM-commands and even leaves those elements of the variable of type "line" undefined that do not need to be defined. With each line a counter "f" is associated, giving the number of PRSYM-commands to be given for that line. Clearing a line now shrinks into setting "f" to zero!
SHORTREP
begin integer k;
line: (integer f; integer array sym[0 : 99]);
lineprint: (k:= 0; while k < f do (PRSYM(sym[k]); k plus 1); NLCR);
lineclear: (f:= 0);
linemark: (sym[x]:= mark;
If f ⩽ x do (k:= f; while k < x do (sym[k]:= space; k plus 1);
f:= x + 1))
end
Note added later.
The above program is essentially the program as I have shown it to at least five different audiences. Now, two months later, while thinking at leisure about correctness proofs, I suddenly realise that the given algorithm for "linemark" betrays my past, for it is a piece of lousy coding, compared with the following alternative:
linemark: (while f ⩽ x do (sym[f]:= space; f plus 1);
sym[x]:= mark)
a version which guarantees that whenever "sym[x]:= mark" is executed, the relation "x < f" will always be satisfied: it is precisely the function of


仙丨=107老师
绿哥师原谅你了 bilibili
六、阅读建议
- 若想通读，先 Dijkstra 第 2、3、7、9、14 章抓主线，再回读 12、13、17 章看虚拟机与递归。
- Hoare 前两章 + 最后一章（公理化）最像"方法论"：中间 3–10 章可当数据结构手册查阅。
- Dahl & Hoare 若不吃 SIMULA 语法，读 1、6、7 章即可理解"前缀=继承"和"概念层次"。
- 三篇都在反抗同一个敌人：**"程序是一长串符号、改程序就是改文本"**的世界观。
需要的话，我可以按 reader skill 的方式，帮你逐篇精读其中某一部分（比如 Dijkstra 的"pearl/necklace"或八皇后），并把关键原图提取出来一起看。
Recap: 把这本书当作"三篇问题散文"而非教材来看——Dijkstra 讲如何用逐步细化和分层虚拟机组织算法，Hoare 讲如何用类型抽象和推迟表示组织数据，Dahl & Hoare 则用 SIMULA 的类、前缀、协程把两者合成概念层次：三条线由"人的智力有限、必须分而治之"这一共同前提串起，最终汇入结构化编程、抽象数据类型与面向对象。
~/Projects/readspace
↑69k ↓7.3k R1.0M CH97.9% $0.035 7.7%/1.0M (auto) deepseek-fla
EDSGER W. DIJKSTRA
1. To My Reader
These notes have the status of "Letters written to myself": I wrote them down because, without doing so, I found myself repeating the same arguments over and over again. When reading what I had written, I was not always too satisfied.
For one thing, I felt that they suffered from a marked verbosity. Yet I do not try to condense them (now), firstly because that would introduce another delay and I would like to "think on", secondly because earlier experiences have made me afraid of being misunderstood: many a programmer tends to see his (sometimes rather specific) difficulties as the core of the subject and as a result there are widely divergent opinions as to what programming is really about.
I hope that, despite its defects, you will enjoy at least parts of it. If these notes prove to be a source of inspiration or to give you a new appreciation of the programmer's trade, some of my goals will have been reached.
Prior to their publication in book form, the "Notes on Structured Programming" have been distributed privately. The interest then shown in them, for which I would like to express my gratitude here, has been one of the main incentives to supplement them with some additional material and to make them available to a wider public. In particular I
notes-on-structured-programming-dijkstra.md 2-31/4862 0%



《结构化程序设计》这类好书，不必只用传统方式读

- 讲者先谈到这本书：它写得很好，已经开始出现一些结构。
- 因此讲者认为，不应该用传统方式读它。
- 他的目标是“我想阅读”，并设想让《结构化程序设计》先帮自己总结全书。
- （整理者串联）这里引出下文：不是放弃读书，而是用 AI 先做全书级总结，降低进入门槛。

今天读书更容易，前提是用对 AI 工具

- 讲者承认：读书不可避免，对大家来说都是如此。
- 但他发现：今天读书比以前容易，前提是用上正确、好用的 AI 工具。
- 他后面用两件事支撑这个判断：用 AI 帮博士生筛经典文献；开车时和 AI 聊 paper。

用 AI 筛 1980–2010 的 systems 经典：结果好到“非常恐怖”

- 讲者给每个博士生的建议都是：需要读一些经典文献。
- 他自述做操作系统、PR（疑为 PL/编程语言）和软件工程相关，认为这三个领域在 **1980–2010 是 golden age**。
- 他的理由是：那之后做的东西越来越小、越来越窄；而 1980–2010 有很多天马行空的想法——在系统还不存在的时候，怎么把这些东西构建出来。
- 具体操作：讲者让 AI bot（转写中作 grog bot / group bot，疑为 Grok bot）在一台云电脑上，收集 1980–2010 年值得读的 systems 文献。
- 结果：AI 找了约 **1000 篇**，大概每个领域两三百篇。

- 讲者基本读过其中大部分文献，于是仔细核对列表；他发现 AI 找的列表非常好，很多文章也是他会推荐的。
- 讲者对此评价是：“非常恐怖。”
- （整理者串联）他没有展开“恐怖”的原因；从上下文看，是因为 AI 的筛选和推荐与他的长期阅读判断高度重合。

开车聊 paper：用 AI 看全文，把 paper 接进知识网络

- 讲者现在开车时会打开 GPT 或 Claude（转写中作 cloud），和它聊一篇 paper。
- 他说 paper 会被索引，AI 实际上能看到全文；他只要把 paper 丢进去，就可以一边开车一边聊“今天这篇 paper 怎么读”。
- 他教过的读 paper 链是：
    1. 从你自己知道的知识开始；
    2. 把领域不停地往外扩张；
    3. 明确你要解决什么问题——可能是一个你原本不知道的问题；
    4. 看它用了什么你原本不知道的手段；
    5. 得到什么结论；
    6. 和你所有知识连成一个网络；
    7. 最终理解这篇 paper。
- 讲者认为，这里面有很多能给人启发的东西。

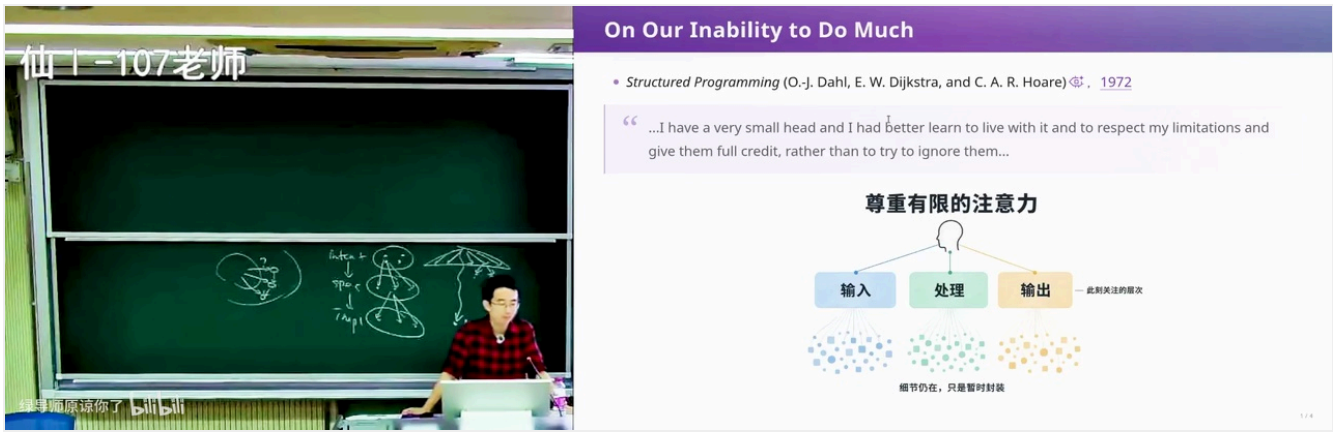
Dijkstra 的提醒：人的能力有限，所以才需要架构与方法

- Dijkstra（转写中作 DASHA）一上来就说 **“our inability to do much”**，即我们无法做太多。
- 承接前面“需求是复杂的，所以我们要做架构”：讲者追问，为什么需要做这件事？他的回答是：**因为人的能力有限**。
- 讲者说，我们就是参数有限的大模型，要承认这一点。
- 他引用：**“I have a very small head, and I had better learn to live with it, and to respect my limitations and give them full credit, rather than trying to ignore [them].”**
- 中文大意：我的脑袋很小，最好学会与它共处，尊重并正视自己的局限，而不是试图忽略它们。

- 讲者认为，这里有很多有意思、有启发的内容。
- 讲者逻辑链：**需求复杂 → 要做架构 → 为什么必须做？ → 人的能力有限 → 我们是参数有限的大模型 → 承认并尊重局限。**

小结：AI 可以放大阅读与理解能力，而 Dijkstra 的提醒把原因拉回人的认知有限性。

第 7 部分





承认“我有个很小的脑袋”：把限制当前提，而不是忽略它

- 讲者引用/认同 **“I have a very small head, and I had better learn to live with it...”**：我有个很小的脑袋，最好学会与之共处，尊重自己的局限，并给这些局限充分认可，而不是试图忽视它们。
- 他由此进入一系列有意思的想法：核心是**抽象加上层次化分解**。

结构化程序设计看似过时，但它处理的是“人不能一次理解”

- 讲者说 **Structured programming** 看起来过时：现在有 **object-oriented**、各种 fancy 框架，前端用 **React 或 Vue**。
- 他调侃 React/Vue 两批社区开发者至今还在“圣战”，像以前 **Vim 和 Emacs** 开发者还在“圣战”，虽然两者越来越像。
- 讲者的逻辑链：**人不能一次理解复杂系统 → 需要层次化分解 → 把不能一次理解的东西变成可以逐层理解的东西**。
- 他的结论：今天所有的这些东西，在过去其实已经被想过一遍。

mental aids：枚举、归纳、抽象是人脑能做的三件事

- 讲者特别喜欢书里的 **mental aids** 部分：**枚举、归纳、抽象**（他提到 part one）。
- 他说，我们人脑能做的就是这三件事情。
- 书里的案例今天看来都是小规模程序；讲者说刚才已经带大家看过。

项链比喻：软件系统是一颗颗珍珠串出来的

- 讲者复述书中的比喻：做一个程序/软件系统，就像做一条 **necklace（项链）**；项链上的每颗小珍珠被串起来，得到一个软件。
- 讲者解释：这个比喻就是在讲**分解复杂性**——把人类不能理解的东西，变成人类可以逐个理解的东西。

教科书脚注与第一手资料：可以忽略，也可以直接追溯

- 讲者设想：老师课上推荐一本书，或教科书脚注说“我的想法来自这本书”。经典 **programming language** 或软件设计书常引用这本书，因为它太有名。
- 讲者说自己读书时基本都面临这个选择：看到“Dijkstra 好像写过这本书”，就**忽略它**；因为把原书挖出来再读，对他来说太浪费时间。
- 但对现在的学生，讲者说：你们可以直接用最原始的第一手信息，按自己想要的方式处理。

1972 年已有 class/object；协程也在构造概念层次

- 讲者指出：书里讲了 **class 和 object**，这是 **1972 年**。
- 接着他谈到**协程**：它考虑的是怎样构造各种各样的概念层次；讲者问“为什么要发明协程？”
- 讲者认为：**协程和线程其实很像**。

线程的比喻：人 + 共享空间

- 讲者上课用过的比喻：**线程 = 人**，**shared memory = 物理空间**。
- 所以人和人之间是协作的；每个人有自己的 memory：我知道自己做到哪一步，今天要干什么，明天要干什么。
- 但同时，我又在这个 shared space 里面。

第 8 部分





私有 memory + 共享空间 → 通信协议 → 复杂性隔离

- 讲者先设定：每个人都有自己私有的 **memory**，知道“我做到哪一步”“今天要干什么”“明天要干什么”。
- 但人在同一个 **shared space** 里，又有共享事项，要合作完成任务。
- 于是讲者断言：人和人之间的 **communication** 必须变成一个 **协议**；协议的作用是把复杂性隔离。
- 逻辑链：
  1. 私有状态 + 共享目标/共享状态同时存在。
  2. 多个执行体/多人需要协作完成整体任务。
  3. 协作不能让每个人、每个执行体都掌握全部细节。
  4. 所以要有通信协议。
  5. **（整理者串联）** 协议固定“谁只关心什么、怎样交换信息”，复杂性因此被切在局部。
- 讲者说，一个工作可分解成多条线；这些线之间通过某个 **数据结构** 或共享物来共享，不断往前推进，并互相通信，最终完成整个任务。
- 他举了 **博弈/下棋**、**编译器（或编译期）** 的例子，并提到一个 **概念层次的金字塔**。这些例子都在说明：复杂系统靠层次、分工和协议管住复杂度。

今天的好设计，其实书里早已讲过；读经典要补回历史语境

- 讲者认为：今天看那些好的设计、好的软件系统，我们做的事情无非是书里已经讲过的。
- 因此可以让 AI 来带我们读。今天假定大家前面东西都多少知道，但讲者说其实你们不知道。
- 例子：**coroutine（协程）**。很多人是操作系统课第一次听说。
  - 在今天的语境下，coroutine 更多是为了性能而实现的 **轻量级线程**。
  - 比如 Go 的协程/goroutine 模型：让你像用线程一样用得很开心，同时每个协程的 **切换** 和 **bookkeeping** 开销都非常小。
  - 原因之一是：你心里已经有线程模型，所以套上去很自然。
- 但讲者强调：如果回到这些机制被设计的基础上、回到那个时间点，当时几乎没有现成的线程模型；它们其实是在应对软件世界里的 **复杂性**。
- 逻辑链：
  1. 今天 coroutine 常被理解成轻量级线程/性能工具。

2. 这个理解依赖后来成熟的线程心智模型。
3. 回到机制诞生时，并没有这种现成模型。
4. 所以不能只用“性能”解释它；它首先是应对复杂性的设计。

- 讲者提议：试一试让 AI 来带读；自己也没有仔细读过，推测是 Part Three、面向上课本科生的部分。让 AI 去读原文，哪怕让它翻译、去掉不太必要的部分，也很好。

抽象 = 把数据 + 操作打包成概念，再逐层搭起来

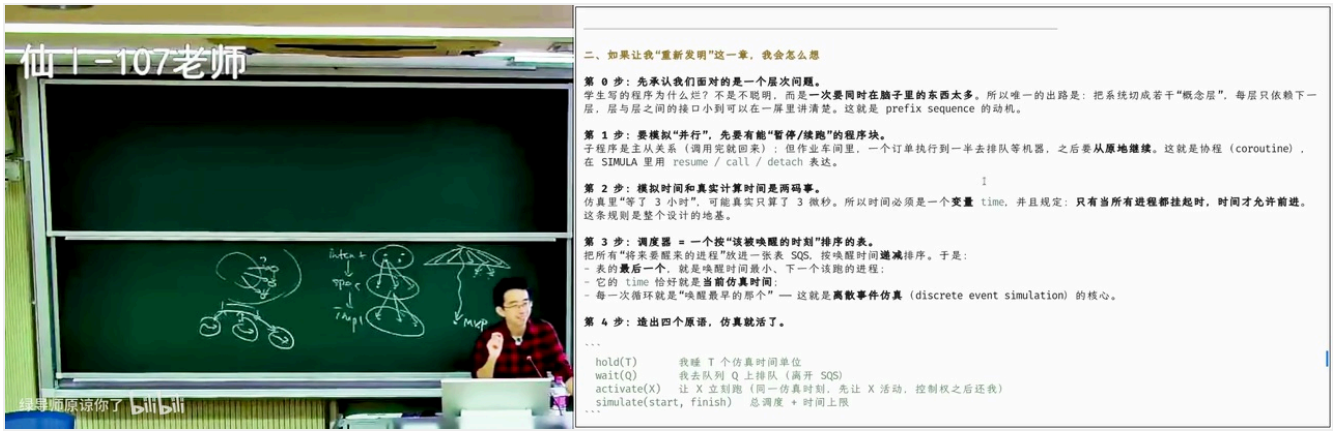
- 讲者转述：把 **数据 + 操作** 打包成一个概念，再用 **[前缀机制]**（按上下文疑为“继承机制”）把概念一层一层搭起来——这就是 **抽象**。
- 模块不管叫什么名字，都是 class 和 method；关键都是把实现内部的复杂性隔离出去。
- 例子：**选课/上课**。
  - “上课”这件事要和系统其他部分在一定程度上隔离。
  - 它可能牵涉 **教室管理**、一个教室、一个课程，以及它们之间怎么关联。
  - 要适当做分类，不能让“上课”直接暴露所有教室、课程、排课细节。
- 逻辑链：
  1. 内部实现总是复杂的（教室、课程、关联关系等）。
  2. 如果外部直接依赖内部细节，复杂度会扩散。
  3. 用概念/类/方法打包数据与操作，给出对外界面。
  4. 外部只和概念打交道，内部复杂性被隔离。
  - **（整理者串联）** 这正是抽象降低认知负担的机制。

学生程序烂的根因：一次要在脑子里装太多东西

- 讲者提到一个观察：学生写的程序为什么这么烂？因为一次要同时在脑子里装的东西太多。
- 他让大家想 OJ 题：最困难的可能不是模拟题，而是需要一点数学性质的题，比如 **动态规划**。
- 如果第一次就对了，样例对了，数据也对了，那还挺好。
- 如果是一两百行的动态规划，然后错了……（转录在此截断）
  - **（整理者串联）** 按前面的“脑内负荷”逻辑，这时错误定位需要同时追踪大量状态，调试难度会急剧上升。

小结：协作靠协议隔离复杂性，抽象靠概念封装降低脑内负荷；今天熟悉的协程等机制也应放回其应对复杂性的历史语境中理解。

第 9 部分



第一次样例/数据都对当然好，但长 DP 一错就极难调

- 讲者开头对比：如果第一次写就对了，样例过、数据也过，那可能还挺好。
- 但如果写了一个一两百行的**动态规划**，然后错了，你根本不知道从哪里调起。
- 讲者认为原因：动态规划程序比其他程序**更难分层**。

模拟能分层并映射到物理状态；DP 的中间层是纯数学表示

讲者给出的逻辑链可以写成：

1. 模拟程序多多少少能把几个大块分开，封装成 class，单独测试、单独调试。
2. 模拟程序的中间状态，很容易映射到一个你能理解的、有**物理含义**的状态。
3. 而动态规划程序中间是**纯粹的数学表示**，两三百行的 DP 一旦出错就是灾难。

所以讲者提出：有没有可能把 DP 的层次或状态改成能以我们熟悉的方式观测?

- 讲者认为，这事情实际上是可以办到的。

写 `DP[i][j][k]` 时，把“状态空间 + 有向无环图”的抽象丢掉了

- 书上写动态规划常写成 `DP[i][j][k]`；写代码时也是 `dp[i]`，然后 `for i`、`for j`、`for k`。
- 只要有一个小地方弄错，整个就全错。
- 错的时候有时不会 crash；crash 反而好找，因为你知道可能越界了。若不 crash，它就算出一个错值，比如转移写错。
- 但当我们谈 `DP[i][j][k]` 时，背后其实有一个问题，一般是一个**优化问题**。
- 这个优化问题本身有**图结构**，可能是图，也可能是别的结构。
- 动态规划本质上是在**状态空间**上做，且有**无后效性**，形成**有向无环图**。
- 这是一层抽象；写 `DP[i][j][k]` 时，虽然思考还在这个抽象层级上，代码却把抽象扔掉了。

把“概念状态”放回代码：可视化、打印迁移，再用 State 替代 i,j,k

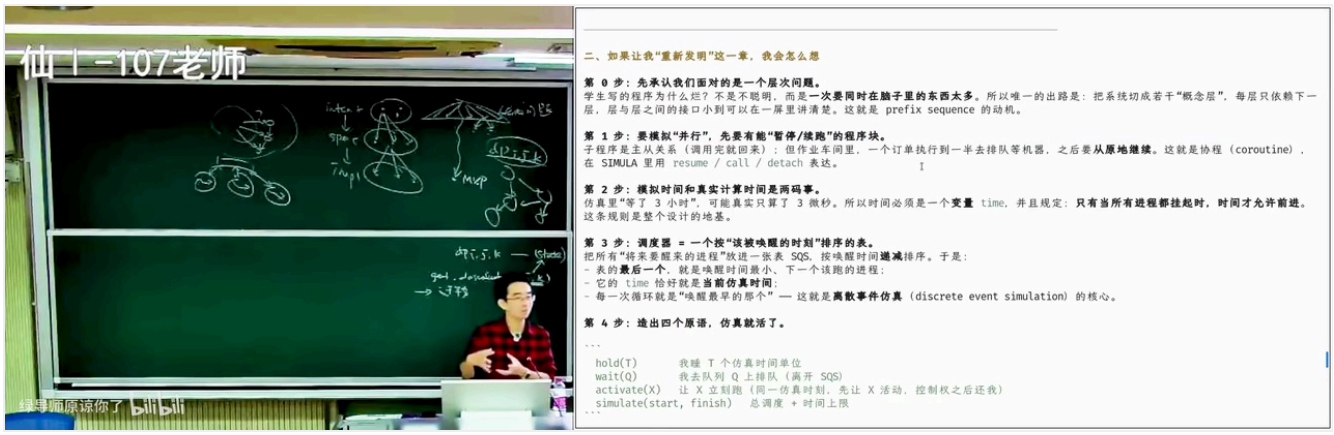
- 可以做的事：把**概念状态**放回来，知道 `DP[i][j][k]` 到底代表什么含义。
- 然后把它**可视化**，甚至把关联状态和迁移打印出来。
- 例如实现一个 `get(i,j,k)`，返回该状态所有可能的迁移。
- 做完这层抽象可能还不够好，可以用更好的抽象，比如用 State 来替代 `i,j,k`。
- 这样一层一层做抽象，程序就可以调试了。
- 调试完之后，再把它改写成**等价的、更高效的形式**。讲者说这只是小例子。

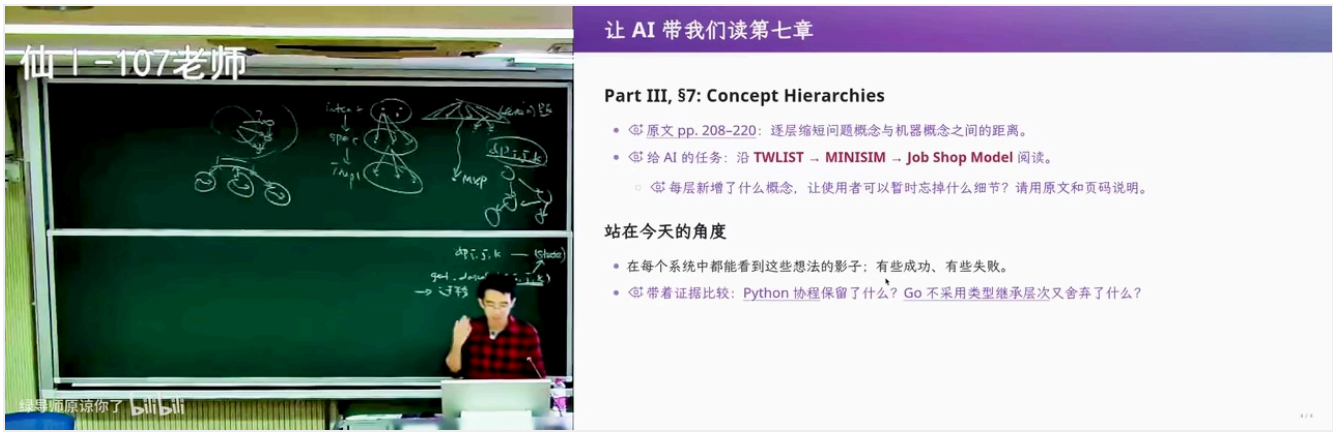
更大粒度：并行模拟可设状态机直接跑，也可做协程抽象

- 讲者扩展到更大的例子：如果要模拟并行，可以创造一个模型。
- 也可以强行直接模拟：设定一个**状态机**，然后直接模拟。
- 还可以做一个**协程（coroutine；原文转写为 CORRATING）抽象**。

小结：DP 难调的根因是代码丢掉了状态空间抽象；先把状态与迁移显式化、可观察，再优化。

第 10 部分





直接模拟 vs coroutine 抽象：合适抽象让“模拟时间”变成另一件事

- 讲者先把模拟分两条路：
  1. **强行直接模拟**：你定义/说有一个**状态机**，然后直接模拟。
  2. **做 coroutine 抽象**：只要做出合适抽象，例如提供 `hold / wait / activate / simulate` 这类原语，就可以去**模拟时间**。
- 讲者判断：有了合适抽象后，“模拟时间”与“直接上手写一个模拟”**很不一样**。
- **（整理者串联）** 这里的差别不在于功能是否一样，而在于抽象是否把时间推进/控制流从手写状态迁移中分离出来。

听不懂就追问；实时语音模型可作为论文的“可追问解释器”

- 讲者说：如果这里没明白，可以继续追问。他因为已有概念，所以知道对方在说什么；学生如果没有，就应该继续问。

- 他最近发现 **ChatGPT 实时语音**很好用：今年开始，GPT 的实时语音模型可以在后台调用一个更强的思考模型，同时保持对话。
- 用法：把论文丢进去；一边问简单问题，它可以立即回答；一边让它想比方，比如“给一个符合现代软件前端的例子”。效果取决于提示词和想象力。
- 例子：更好的模型找到了 **React Scheduler 和 Fiber 架构**的对应。
- 讲者说不知道 DPC（录音如此，疑为某模型名）会给什么，但如果用更好的模型，就能得到这种对应。
- **（整理者串联）** 这段是在给跨过理解障碍的方法：不是死磕原文，而是把材料变成可追问、可要求类比的交互对象。

有 web search 的模型能从大量论文中挑出“好抽象”的经典例子

- 讲者把问题再上升一步：不谈具体例子，只问**抽象的想法**有什么。
- 他认为最经典的**预训练模型没有 web search**：只能从“脑子里”的直觉找答案。
- 如果有 web search，就可以要求 GPT 去调研 **100 篇/1000 篇论文**，或者从他的“池子”里的约千篇论文中，选一篇最好的、最符合要求的。
- 结果找到的例子包括 **UNIX**、**关系数据库**、**interface**、**protocol**、**TCP/IP**；讲者说这些都在告诉你要做一个**好的抽象**。
- 这些例子他今天上课都会讲。讲者觉得，某种程度上“我们还是达成了一定的共识/一致”。

课程不可能覆盖全部；但启发是“读书也许没那么困难”

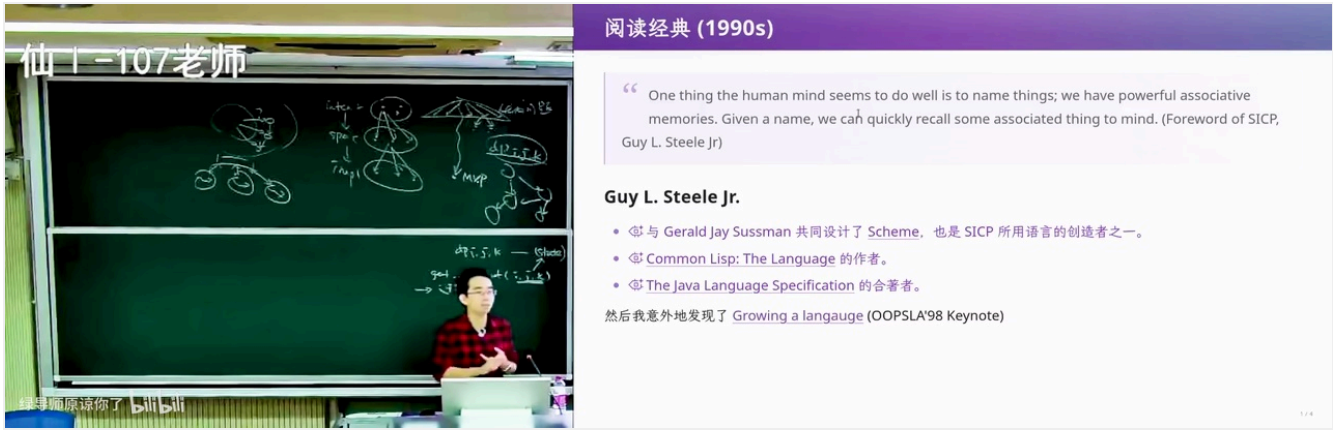
- 讲者认为，这门课讲什么都不够：他无法在一门课里覆盖那 **1000 篇**论文。
- 如果在他这里读 PhD，比如做 **system software**，这些论文多多少少都得读，这是很恐怖的事情。
- 但学生得到的启发是：**读书也许没有想象中那么困难**。
- **（整理者串联）** 这个启发来自前面用 web search/实时语音模型调研、筛选、类比论文的做法：工具降低了从大量文献中定位相关抽象的成本。

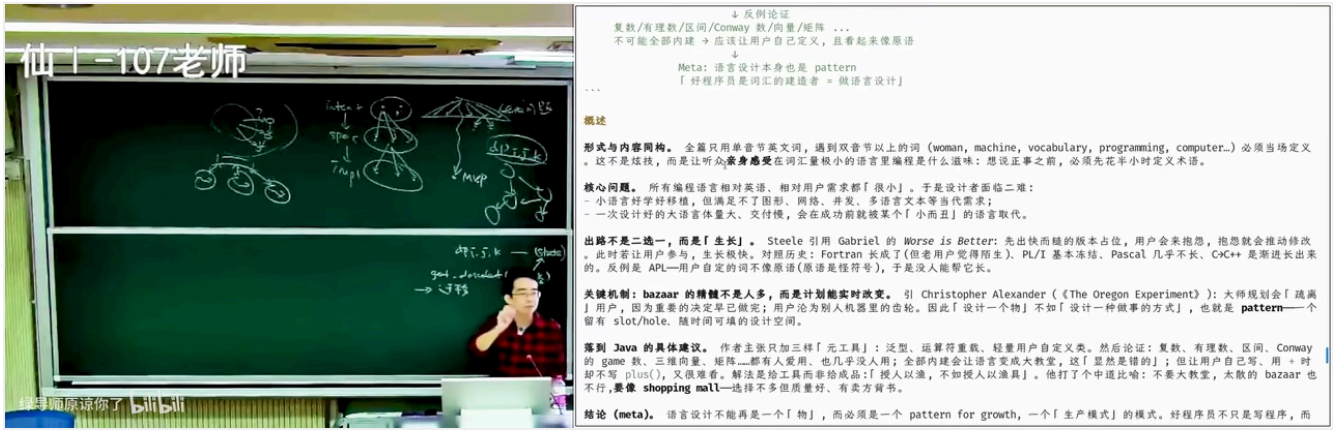
回到 1980/1990：找到 Scheme 设计者、Java 规范合作者的 OOPSLA talk

- 讲者在准备今天课时又找了一些有点老的材料。刚才是 **1972 年**；他接着问：到 **1980 年、1990 年**，又怎么看这件事？
- 他找到一个很有意思的 talk。讲者是 **Scheme 的设计者**，也是 **Java Language Specification** 的合作者。
- 讲者自己读过 **Java Language Specification** 和 **Java Virtual Machine Specification**；这些规范会告诉你语言和虚拟机是怎么设计的。
- 这个 talk 是在 **OOPSLA**（录音作“乌普斯达酒吧”；整理者校正）的时候讲的。
- **（整理者串联）** 从这里开始，讲者把时间线从 1972 推向 1980/1990 年代，准备用这位语言设计者的 talk 继续讨论“好抽象/语言设计”。

小结：讲者这条线的落点是——关键不是覆盖所有文献，而是获得能自己寻找、类比并识别好抽象的路径。

第 11 部分





从《Java Language Specification》到 OOPSLA '98：这篇 Growing a Language 值得停下来读

- 讲者先提到《Java Language Specification》和《Java Virtual Machine Specification》：它们说明语言和虚拟机如何设计。
- **Growing a Language** 的作者 Guy Steele 在 **OOPSLA '98**（原转写“乌普斯达酒吧”）做 keynote；OOPSLA 是 programming language / object-oriented 方向的顶级会议。讲者称他当时已功成名就，是当时最火语言之一的设计者。
- 讲者当场打开 PDF。第一眼看到的是 **"Most of the time I do not read my talks"**；如果只是翻 PDF，会觉得十几页全是字，然后关掉。讲者说应该停下来：这场 talk 非常有趣。

全单音节 + 现场定义多音节词：这是表演性论证，不只是炫技

- 这是 **performative speech**。他说 **"I have to read"**，要把它读出来。
- 规则：全篇只用**单音节英文词**；遇到双音节及以上词，如 **woman, machine, vocabulary, programming, computer**，必须当场定义。
- 讲者类比写藏头诗或全单音节约束写作，认为这非常炫技。
- 讲者评价：这种品味和 **AI 的输出**不一样；AI 写不出这种东西；至少从今天的角度看，要想到这件事还需要**人类创造力**。

language 是双关：语言要能被使用，就必须能生长

- 他用这个方式讲 **Growing a Language**。这里的 **language** 既指英语/自然语言，也指 programming language。
- 起初全用单音节英文词；随后会发现这种语言很难用——至少对人类很难用。想定义复杂事物、想把任何东西讲清楚，都会变得很复杂，遇到瓶颈。

- （整理者串联）这把“写作约束”引到“语言设计”：语言若不能生长、不能引入并定义新概念，就会难用。

可执行 → 形式语义 → 编程语言不能像自然语言那样大 讲者的链条：

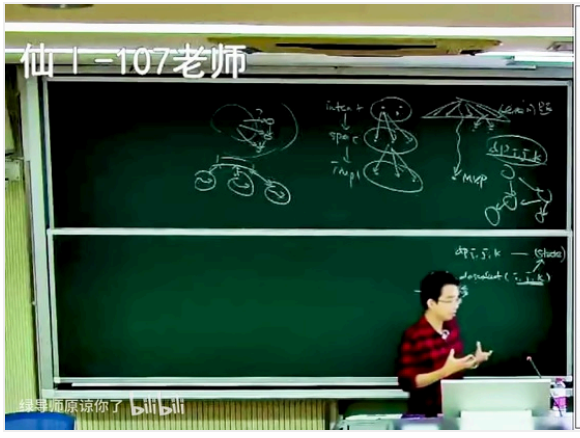
1. 所有编程语言相对于英语、相对于用户需求都很小。
2. 原因：编程语言需要 **formal semantics（形式语义）**。
3. 如果语言变得非常大，里面有像 **woman** 这样的概念，它的形式语义是什么？定义不出来。
4. 必须有形式语义的东西才能在计算机上执行。
5. 所以编程语言相对而言都比较小。

极简设计不是答案：Brainfuck 式语言会小到无法写、无效

- 如果编程语言按 **minimal design** 做到最小最小，例如 **Brainfuck（原转写只出现 brain）**，那就完全无法写程序，也没有效。
- （整理者串联）因此这里的对比是：语言不能像自然语言一样无限膨胀，但也不能小到失去可用性。

小结：讲者这一段的论证是：可执行要求形式语义 → 编程语言不可能像自然语言那样大；但极简到 Brainfuck 又不可用。

第 12 部分





Meta：语言设计本身也是 pattern
「好程序员是词汇的建造者 = 做语言设计」

概述

形式与内容同构。 全篇只用单音节英文词，遇到双音节以上的词（woman, machine, vocabulary, programming, computer…）必须当场定义。这不是炫技，而是让听众**亲身感受**在词汇量极小的语言里编程是什么滋味：想说正事之前，必须先花半小时定义术语。

核心问题。 所有编程语言相对英语¡ 相对用户需求都「很小」。于是设计者面临二难：
- 小语言好学好移植，但满足不了图形、网络、并发、多语言文本等当代需求；
- 一次设计好的大语言体量大、交付慢，会在成功前就被某个「小而丑」的语言取代。

出路不是二选一，而是「生长」。 Steele 引用 Gabriel 的 *Worse is Better*：先出快而糙的版本占位，用户会来抱怨，抱怨就会推动修改。此时若让用户参与，生长极快。对照历史：Fortran 长成了（但老用户觉得陌生）、PL/I 基本冻结、Pascal 几乎不长、C>C++ 是渐进长出来的。反例是 APL——用户自定的词不像原语（原语是怪符号），于是没人能帮它长。

关键机制：bazaar 的精髓不是人多，而是计划能实时改变。 引 Christopher Alexander（《The Oregon Experiment》）：大师规划会「疏离」用户，因为重要的决定早已做完；用户沦为别人机器里的齿轮。因此「设计一个物」不如「设计一种做事的方式」，也就是 **pattern**——一个留有 slot/hole、随时间可填的设计空间。

落到 Java 的具体建议。 作者主张只加三样「元工具」：泛型、运算符重载、轻量用户自定义类。然后论证：复数、有理数、区间、Conway 的 game 数、三维向量、矩阵……都有人爱用、也几乎没人用；全部内建会让语言变成大教堂，这「显然是错的」；但让用户自己写、用 + 时却不写 plus()，又很难看。解法是给工具而非给成品：「授人以渔，不如授人以渔具」。他打了个中道比喻：不要大教堂，太散的 bazaar 也不行，**要像 shopping mall**——选择不多但质量好、有卖方背书。

结论（meta）。 语言设计不能再是一个「物」，而必须是一个 pattern for growth，一个「生产模式」的模式。好程序员不只是写程序，而是在搭建一套可用词汇——即在做语言设计。他还提醒：wart 有「可日后拔除的 wart」，也有「拔不掉的 wart」；有些好特性是「暂时故意留空」的，要保证未来能补上。

极端最小语言能编码图灵机，但人类不可读

- 讲者以 **Brainfuck**（转写作“brain”）为例：没用过它的人完全写不出程序，它也不有效。
- 讲者说它基本上就是把**图灵机**直接编码成语言：语言是一串一串符号，有一个指针/指代，可以往左、往右，做加法、判断/循环。
- 它能写出 **Hello World**，满足“最小”，但代价是**人类不可读**。所以实际编程语言不会那么离谱，仍然要可用。

C 贴近机器而成功，但抽象能力不足会反噬表达

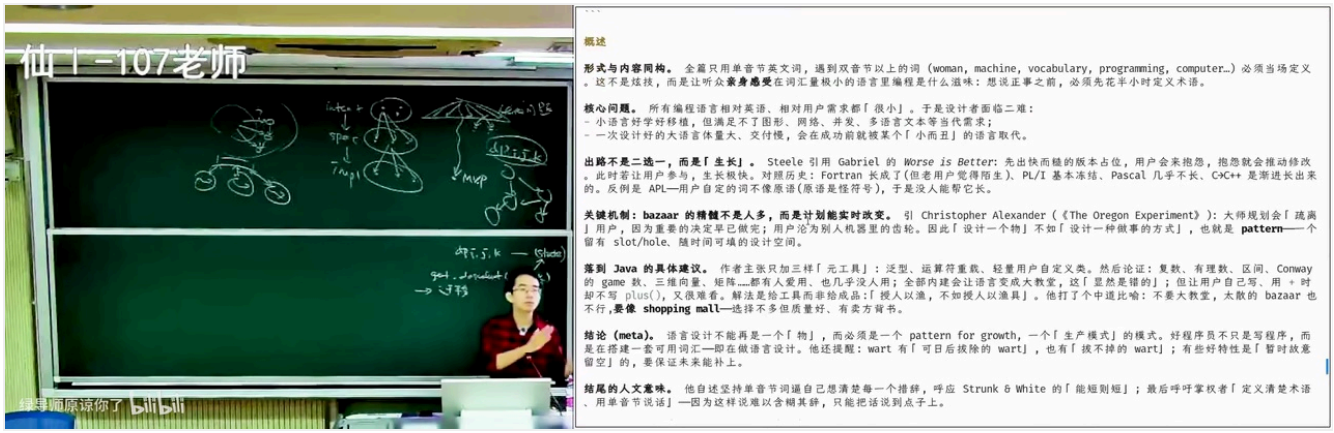
- 讲者认为 **C** 是设计得非常经典的可用语言，但也带来设计问题。
- 例一：C 没有**运算符重载**。在 C 里表达“往队列里放东西”非常恐怖（若 C 写得多）。
- 例二：**qsort** 这类 API 设计糟糕：比较函数指针签名涉及 `const void *`，还要数组、`nmemb`、`size` 等参数。讲者说 C 没有能力让用户自定义比较好的语言抽象，所以表达任何东西都困难。
- 讲者用比喻说：这就像“用单音节词去定义双音节词”时遇到困难。
- 逻辑链（讲者明确给出）：
  1. 小语言 → 好移植、好用，但满足不了不断膨胀的需求；
  2. 一次设计好的大语言 → 体量大、交付慢 → 成功前被某个小而丑的语言取代；
  3. 所以 C 是典型成功者：很大程度因为它**直接和机器一一对应**、没有额外抽象，几乎可移植到任何地方。
- 讲者补充时代条件：在人力写代码的年代，写编译器非常复杂；在这个意义下，这形成了一种编程语言的架构。

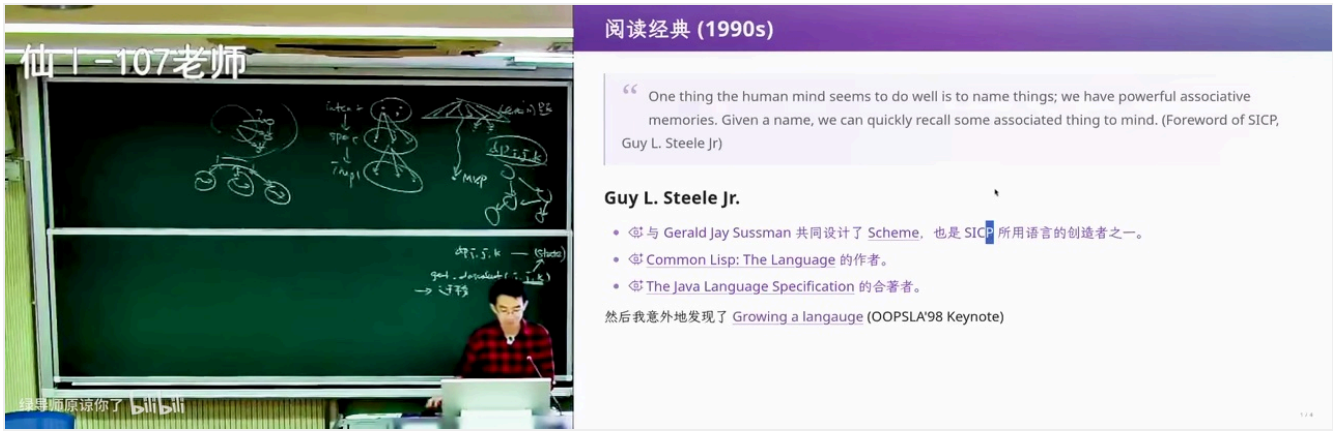
编程语言里有什么，本身就是架构；出路不是二选一，而是生长

- 讲者扩展“架构”：人们谈架构时常想到用什么数据库、如何设计 class 表示 **Student**、**Course**；但这些只是软件架构的一部分。
- “**你的编程语言里有什么**”本身也是架构。前面 **QR code/二维码**的例子——创建一个小语言——也是架构。
- 这个架构提出的深刻问题是：**出路不是二选一，而是生长**。
- 讲者接回第一次课：好的软件/软件设计能抵抗需求变更；当需求变化、世界变化时……

- （整理者串联）因此语言设计不能只在“最小”和“大而全”之间选边，而要让语言/架构随需求变化生长。

第 13 部分





好架构要能抵抗需求变更，并给未来生长留路径

- 讲者回顾第一次课的观点：好的软件 / 软件设计能够抵抗需求变更。
- 当需求变化、世界变化时，设计者只能在一定程度上前瞻性地看到未来会发生什么。
- 如果只针对现在设计，没有考虑软件应该怎样生长，就可能陷入**技术债**的泥潭。
- 讲者认为：具备**生长能力**的架构是最好的。

讲者给出的逻辑链：

1. 需求和世界会变化。
2. 如果设计只满足当下，就没有给未来变化留下生长路径。

3. 变化到来时，系统容易积累技术债。
4. 所以，架构需要具备生长能力。

C→C++ 的案例：能生长不等于长得好

- 讲者举了一些老语言为例，提到 **Pascal（帕斯卡）**，并说自己用过；他认为听众应该都没用过。
- **C 到 C++** 是一个“长出来”的例子。
- 但讲者认为：**C++ 长得并不好**。
- 以**模板**为例：
  - 模板是非常重要的架构机制。
  - 但在 **C++11 / 14 / 17 之前**，为了实现各种功能，模板被滥用。
  - **Boost** 里那些模板没人能看懂；尤其对学生来说，大家不知道那是什么，需要花很长时间才能理解。
  - 讲者判断：模板本身是很好的机制，但长出了比较奇怪的东西。
- （整理者串联：这说明“能长”本身不等于“长得好”；生长若失控，会带来理解成本和滥用。）

给 Java 的“只加三样”建议：泛型、运算符重载、轻量用户自定义类

- 讲者转述“他”给 Java 的建议：只加三样。
  - **泛型**很重要；没有泛型，构造抽象时会像 C 或早期 Java 那样，到处都有强制类型转换。
  - **运算符重载**；讲者说 Java 至今好像没有加入。
  - **轻量用户自定义类**；讲者说现在已有。
- 讲者由这些语言 / 架构故事进一步说：每个同学都有自己的偏好，问出来的问题不一样，结果也会不一样。

可生长架构的哲学：扩展是一等公民，并且可组合

- 讲者把正在讨论的这场演讲概括为一条关于 **growability（可生长性）** 的架构哲学。
- 核心主张：
  - **扩展必须是一等公民**。

    - **扩展之间可以组合**。
- 这才是可生长的架构。
- 反面：不是“我有一个需求清单，然后把需求清单全部实现”，这就叫正确。
- 更准确地说：扩展必须是一等公民，语言要提供组合机制，而不是**用例清单**。

OJ 把学生训练成解题机器，削弱了扩展与维护意识

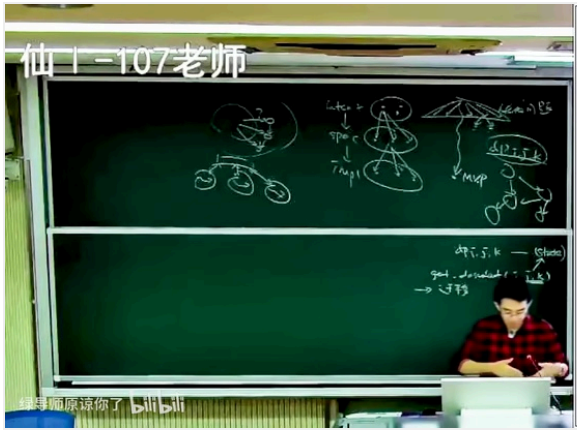
- 讲者指出，做 OJ 有一个影响：从进大学开始疯狂做各种 OJ，会把学生训练成**解题机器**。
- 原因：做 OJ 不需要任何扩展；题对了就行，代码不需要维护。
- 这会带来一种不自觉的倾向：写**用完即扔**的代码。
- 后果：学生在这方面的思考变少，也就是对扩展、架构、维护的思考变少。
- 讲者认为，这门课会把这方面拉回来一点。
- 再次强调：**扩展必须是一等公民**，要提供语言和组合机制，而不是用例清单。

从需求清单转向概念与组合

- 可以照着需求清单去写实现。
- 但如果把需求分解成“跳出问题、看问题的几个概念”，这些概念又能变成具体的、领域里的东西，并且可以组合，那可能就是对的 / 好的设计。

小结：讲者的判断是，扩展必须是一等公民且可组合；只按需求清单实现只是“正确”，不是可生长。

第 14 部分



对应到软件架构，就是框架设计里经典的 **frozen spots / hot spots**（Parnas、Fayad & Schmidt 的框架理论）：框架冻结不变的部分（控制流、生命周期、协议），把变化的部分留成扩展点。架构的好坏不取决于它今天覆盖了多少功能，而取决于**它把哪些决定推迟了、推迟得是否优雅**。

2. 扩展必须是「一等公民」

APL 的死因是「用户自定义的东西不像原语」：

> *"in APL, new words defined by the user do not look like language primitives at all... This has stopped users from helping to grow the language."*

架构含义很强：**插件的体验必须和内置机制同质。**如果扩展只能走 `plugin.doThing()` 而内置走 `+`，用户就会觉得在写二等代码，生态就长不起来。这也是为什么 Kubernetes 的 CRD、VS Code 的 extension API、Rust 的 trait 都要做到「用户定义的类型/资源在第一层语法和工具链里不特殊对待」。反过来说：**任何需要用户"绕过"你的抽象才能用的扩展机制，都是架构缺陷。**

3. 给原语和组合机制，而不是用例清单

这是演讲后半段的正面论证。复数的加法、Clifford 数、区间算术……Steele 逐一追问「要不要内建？」最后：

> *"I might say 'yes' to each one of these, but it is clear that I must say 'no' to all of them!"*

因为「内建」等于把 niche 需求让所有人买单。他给的替代是：加**还型 + 运算符重载 + 轻量数值类型**这三样「元能力」，其余交给库。

架构上这就是 **platform vs product** 的分野：不要为每个业务方定制一条路径，提供抽象 + 组合子；让 niche 需求以库的形式存在。而且他还指出了关键收益——「a few and then put up as libraries for other users to use, or not, as they choose」：**可选性本身就是价值**，内建则强加给所有人。

4. 关键不是人多，而是计划能实时改变

> *"the key point is that in the bazaar style... the plan can change in real time to meet the needs of those who work on it and use it."*

读论文快慢取决于已有知识：从已知往外推，AI 降低整理成本

- 讲者先提到一种判断方式：读到领域里的东西时，如果能判断它可以组合、可能是一个对的/好的设计，他可能就不再继续看下去了。
- 讲者认为，大二、大三刚开始读论文/读这些东西时，一定非常慢，因为学习方式是“从你已知的知识开始往外推”。
- 逻辑链：**已有知识越多 → 新东西越容易挂到已知结构上 → 读得越快**；反之，核心知识不足时就会读得慢且痛苦。
- 讲者自述：他现在读论文很快，是因为已知知识相对较多，而且 **AI 可以帮他整理**。
- 但在建立一个比较好的 **Computer Science 核心知识**之前，会经历一个比较痛苦的过程：你不理解为什么。
- 好消息是：可以问 AI 各种各样的事情。
- **（整理者串联）** AI 能降低查询、整理的成本，但讲者这里并没有说 AI 能替代核心知识的建立；从他仍强调“建立核心知识之前会痛苦”来看，核心知识仍要自己长出来。

快速发展的时代埋没经典，品味被人类 slop 和 AI slop 污染

- 讲者的 comment：**有太多宝藏被埋在这个快速发展的时代里面了**。
- 进组后，大家常听到类似说法：“我们最近又发表了一篇 state of the art 的文章”，“师兄又发表了一篇 ‘self’ 的文章（原词不清）”。
- 讲者认为，某种程度上大家的品味被海量的 **人类 slop** 和 **AI slop** 污染了。

- 他举的例子包括：**CSDN 上的东西**、妨碍自学的教科书。并说今天举的两个只是非常随意的例子，这些经典可能有成百上千。
- 讲者自述：从本科高年级到博士毕业这段时间，他不可能把这些经典都读完，只读了其中一部分。他自认算是读书读得比较多的人，也只是很少的一部分，并有一些自己的想法。
- 但在今天，每个人都可以快速、而且原汁原味地去了解作者想要传达的原始想法。
- 因此讲者非常期待一个**“大学毁灭计划”**：有没有可能从大一第一门课 **Programming Language** 就把知识点全拆散；不用一本教科书，甚至也不用上课老师写的教科书；把原始知识全部 **break down**，把 1970、1980、1990 年代的经典全部拆开，再重新串联起来，为你生成一本教材。讲者觉得这是一个挺好玩的事情。

架构应可生长；Unix philosophy 用文本流统一接口

- 讲者转到架构：他从 *Growing a Language* 中提炼出观点：**一个能够生长的架构才是最好的架构**。
- 架构设计的空间有哪些经典案例？第一个就是大名鼎鼎的 **Unix philosophy**。
- 讲者给出三条原则：
    1. 每个工具 **Do one thing, do well**。
    2. **Write programs to work together**。
    3. **Write programs to handle text streams**。
- 讲者认为第三条是非常精髓的设计：**文本接口统一了人和程序、程序和程序的接口**。因为如果只是 binary 的字节流，对人来说是很不友好的。
- 数据库/SQL 的对比：你们都知道数据库好，但用数据库时总觉得隔了一层纱，因为每一次查询任何东西都需要一个 **SQL query**，而这个 SQL query 对你是 **indirect** 的；而你只是想要一个立即的东西。

第 15 部分





人觉得“隔了一层纱”：每次操作都加间接层会让人痛苦

- 讲者先讲一种体验：你每次查询任何东西都要写一个 **SQL query**；这个 query 对你是 **indirect** 的。
- 你真正想要的是立即得到的东西，所以人不适应在做任何事时都额外加一层 **indirection**。
- 模型可以遵循这种间接规则：比如让它每说一句话后都说“喵”，它能做到。
- 但如果要求人来遵循这种规则，人会觉得非常痛苦。

逻辑链：
用户想要直接结果 → 每次却都要经过 SQL query → 多出一层间接 → 人觉得隔了一层纱、很痛苦。
（整理者串联） 这为后文 UNIX 的统一接口与可组合机制埋下对照：好的机制应减少不必要的间接摩擦。

原文第一要点：像接花园水管一样连接程序到管道

- 讲者提到，当你有一个 AI 帮你收集资料，并且能帮你总结原文时，就能看到原文的四个要点。
- 第一个要点是：**像接花园水管一样连接程序到管道**。
- 讲者说，这个想法在 **1964 年**就已经出现；管道应是在 **1973 年**（转录作 1983）引入 UNIX。
- 后面还有许多关于操作系统设计的经典想法，如**动态链接**、**库的索引**等；讲者认为它们对今天的系统设计有深远影响。

UNIX 的统一标准让命令行工具可被人和工具共同理解

- UNIX 依靠一套“UNIX 世界里的语言”。
- 有人（如《Unix Haters Handbook》，转录作“unix hater sky”）吐槽 UNIX 设计不好。
- 但讲者强调：它有一套**统一标准**——所有命令行工具遵循大致一样的**命令行参数**，所以人很好理解。
- 输出格式也尽可能保持一致；当 **sed**、**awk** 等几大工具奠定技术以后，程序输出开始朝对这几个工具友好、同时也对人和工具友好的方向继续发展。
- **退出状态（exit status）** 能在程序出问题时，及时告诉另一个程序。
- 讲者总结：这些加上**管道**，就是一个非常棒的机制。

打开文件菜单的例子：普通软件只能二选一，UNIX 可以自己配

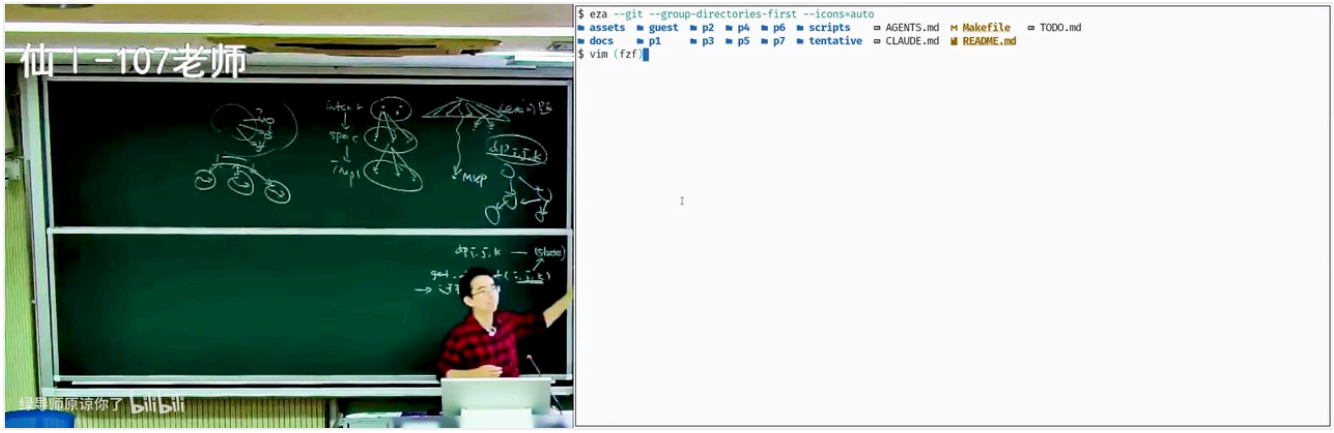
- 假设你要设计软件，需要一个“打开文件”菜单。通常要么自己实现，要么使用操作系统的。
- 讲者问：如果对这两个都不满意，你有办法吗？在常见情况下，没有办法。
- 例子：学校买的 **Pr**（讲者说属于 Adobe 全家桶）里的“打开”好像是 Adobe 自己的菜单，因为它可能提供预览等功能；好用的时候挺好用，不符合使用习惯时很难用。
- 操作系统的打开文档窗口也不是那么好用，虽然它集成了绝大部分常用功能。
- UNIX 里的答案不一样：比如用 **Vim**、**Emacs**、**VS Code** 等任何一个，或者用 **nano**、**Neovim**（转录作 WM、IMAX、不胜战、NANO、NEO；按上下文校正），都可以给它配一个自己的菜单。
- 讲者展示：他配出了一个自己的“打开文件”菜单。

逻辑链：

普通软件中，打开菜单由应用或 OS 固定 → 用户不满意也无解。

UNIX 中，统一接口 + 可组合工具 → 用户可以自己配菜单、替换组件。
（整理者串联） 所以 UNIX 的机制把“打开文件”从不可改的固定层，变成可由用户接管的可组合层。

第 16 部分



UNIX 的文本接口把工具变成可无限组合的小程序

自定义菜单示例：组合性体现在“我能配自己的菜单”

- 讲者提到用 NANO，或 NEO（原转写，编辑器名存疑）：想给它配一个菜单，于是配出自己的菜单，最后得到一个“打开文件”的菜单。
- 讲者称这就是**工具的组合性**，也体现模块化。
- 讲者说 UNIX 的这个结果有偶然性：系统设计者当时未必想到后来这么多事情。
- 具体例子：讲者用 NANO 打开“第一课的 index”。

shell、编程语言、管道为什么是一脉相承

- 讲者说：**shell（原转写 share）的文本接口**和**编程语言**、**UNIX 管道文本接口**是一脉相承的。
- 把它们放在一起，就有**无限组合和复用的空间**，允许你写程序。
- 讲者给出的逻辑链：统一文本接口 → 工具可组合 → 用户可写程序、拼流程。

讲者判断：用 UNIX/Linux 的人从敲命令起就是程序员

- 讲者在操作系统课上常说：你们都是程序员。
- 从用 Linux/UNIX 的那一刻起，每个人都是程序员：因为**每一个命令都是一个小程序**。
- 这些小程序的职责，是把各种 do one thing and do it well 的工具组合起来。
- 讲者认为这种**可组合性非常关键**。
- 讲者认为：到今天都很难在一个 **GUI 程序**（原转写 GUE）里得到这样的组合性；这种 ground breaking 的东西应该是可以做得出来的，可以畅想。
- 讲者收束：这是 UNIX。

关系数据库：用二维表模型换统一管理，同时保住接口的可组合性

文件系统管理数据：raw interface 的得与失

- 第二个熟悉例子是**关系数据库**。讲者说这个例子已经讲过几次。
- 讲者引用 E.F. Codd 1970 年论文的说法：

> “A relational model of data ... is a basis for protecting users of formatted data systems from the potentially disruptive changes in data representation caused by growth in the data bank and changes in traffic.”

- 如果直接用**文件系统**管理数据，面对的是 **raw interface（裸接口；原转写 row interface）**：
  - 好处：UNIX 世界里的工具可以复用。
  - 坏处：太自由，没有统一的管理。
- 讲者给出的逻辑链：文件系统 → raw interface → UNIX 工具可复用；坏处是太自由、无统一管理。

E.F. Codd 1970：把世界建模成二维表和表间操作

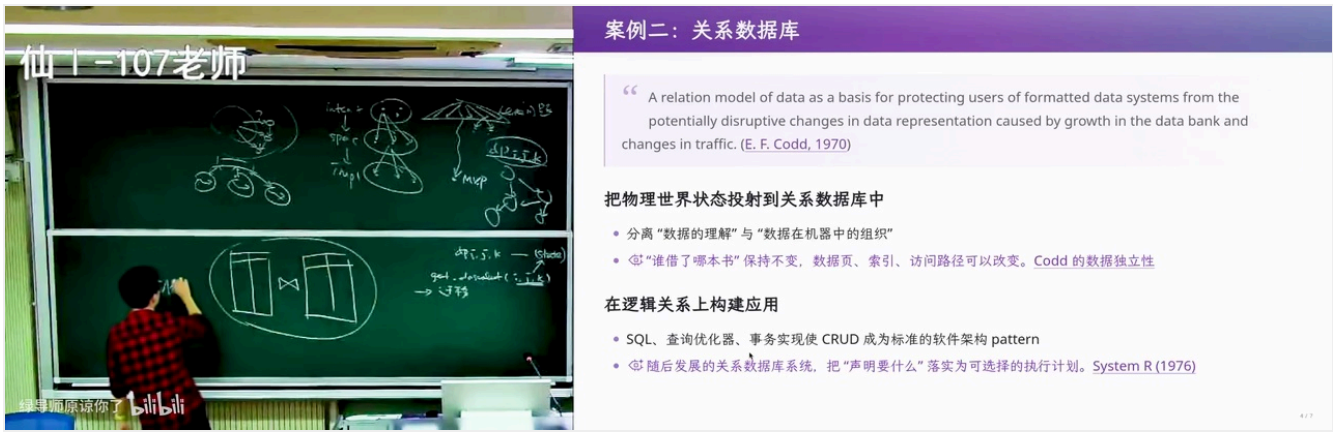
- 讲者认为关系模型/关系代数非常了不起。
- 讲者个人经验：自己做过一段时间**程序合成**；凡是做与人类世界相关的程序合成，绕不开 E.F. Codd 1970 年论文。
- 该论文对世界建立模型：世界里的数据可以用**二维表**表示。

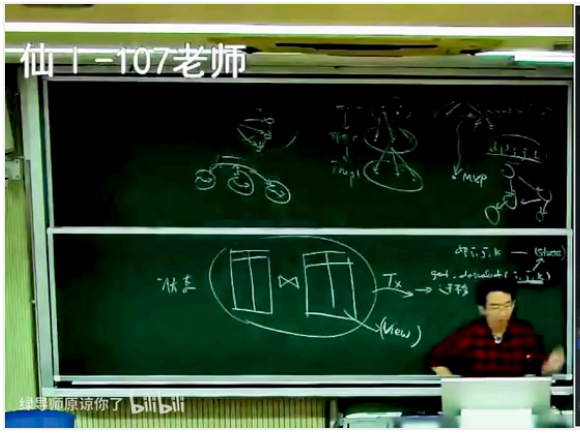
- 有好多表，以及表与表之间的操作；几乎可以用这些表间操作（如 **join**，原转写为 drawing，存疑）来处理所需的数据管理。
- 至于数据怎么管、怎么存、可靠不可靠，全部交给底层**数据库系统的实现**。
- 讲者给出的逻辑链：二维表模型 → 表间操作表达数据管理 → 存储/可靠性下沉到数据库系统实现。

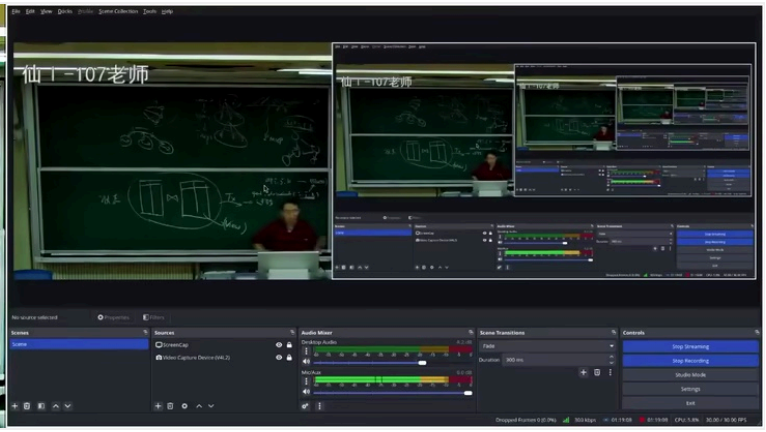
SQL 是接口/语言：有了 SQL、优化器和事务，数据库成为状态机

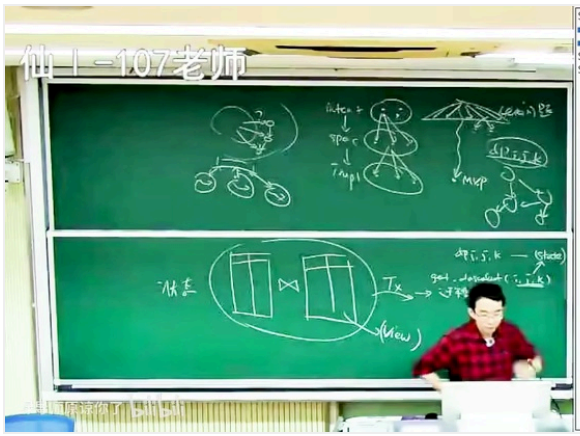
- 在关系模型/关系代数基础上，有了 **SQL（结构化查询语言）**。
- 讲者并置：UNIX shell/bash 是一个语言，SQL 也是一个语言。
- 讲者判断：当一个东西可以 grow 时，它多多少少会带上**语义的性质**；它是一个**接口**，接口足够灵活，就会像一个**编程语言**。
- 有了 SQL，然后有了**查询优化器**；再有了**事务**以后：
  - 数据库在任何时候的**二维表都是当前世界的一个状态**。
  - 这个状态经过一次数据库 **transaction** 会到下一个状态。
- 数据库还可以有**视图（view）**：当前状态可以用一个查询来创建。

第 17 部分











数据库状态与视图：transaction 改状态，view 用查询表达当前世界

- **状态经过一次数据库 transaction 会到下一个状态。**
- 数据库还可以有**视图（view）**：当前状态可以用一个查询来创建，也就是“我对当前世界的理解”。
- 数据库系统的实现给了**近乎无限 scale 的能力**；讲者说，到今天 SQL（原转录作 SEO）的数据依然可以无限 scale。
- 因此，可以用这样的信息系统。

关系型数据库：把实体间逻辑关系理清，就能在现实世界实现信息系统

- 讲者说，数据库带起了**整个人类世界数据信息化的一波大浪潮**。
- 例子：图书馆里的书、学籍管理系统、财务信息、在线商城订单都可以信息化。
- 所有东西都可以用 **relational database（关系型数据库）** 表示：它是逻辑关系，表示**实体与实体之间的逻辑关系**。

- 讲者的推理：只要把这个逻辑关系理清楚，你就可以在现实世界里面实现信息系统。
- 讲者评价：这是非常顶级的、**隔离了复杂性的抽象**，而且有**无穷的扩展能力**。
- 扩展能力的关键：它只规定这个世界里有表/数据库的 **schema**（原转录作“表DELBASE的schema”）；但 schema 是什么，可以由你扩展、由你决定、为这个应用定制。

数据库逻辑链：状态经 transaction 更新 → view/query 表达当前世界 → DBMS 实现近乎无限 scale → 可用信息系统 → 关系型数据库表示实体间逻辑关系 → 理清逻辑关系即可在现实世界实现信息系统 → schema 由应用定制、扩展无穷。

Internet：先把世界连起来，且架构要能成长/变化

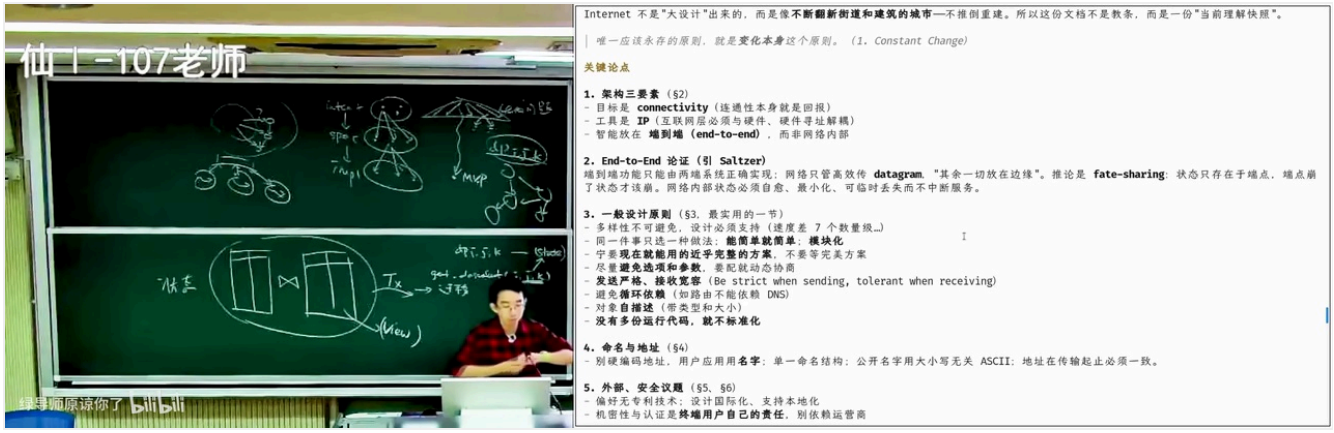
- 讲者说 Internet 也是非常典型的案例，解决一个非常简单的问题；他认为这也是非常非常经典的案例。
- 他用自身场景说明：今天为什么能站在那上课，同时还在推流，推流时手机还能看推流结果和弹幕？——因为 Internet 把整个世界连起来了。
- 这个“连起来”是一个**可以成长的架构**。
- 它负责解决系统里一部分问题：例如用 **TCP** 时，你不用担心数据传输到底可靠不可靠；任何数据流都可以稳定可靠地从一点送到另一点，从手机到世界另一头的某个点都没问题。
- 上面跑什么应用程序，就支持什么；**不要做预设**。
- 讲者推荐读 **RFC 1958, Architectural Principles of the Internet, 1996 年 6 月**。
- 他认为核心原则是：**唯一应该永存的原则就是变化本身**——架构要承载世界的变化。

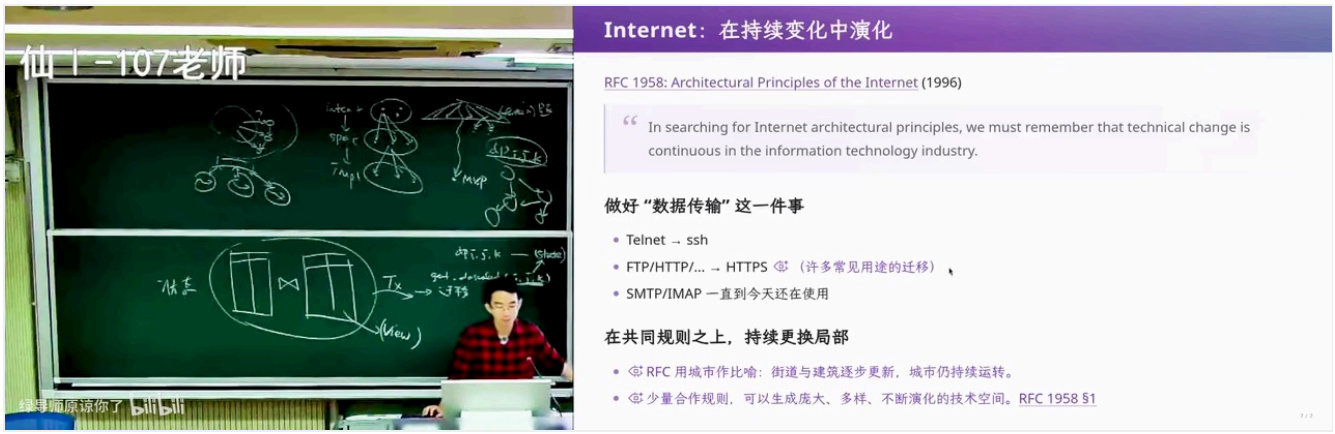
Internet 的设计选择：目标就是 connectivity，其他交给上层/别人

- Internet 只做一件事：**networking**；目标就是 **connectivity**。
- 在 connectivity 的基础上，上面会有很多应用爆发出来；但它不管。
- 例子：现在有些数据中心还在用 **TCP/IP**，虽然有更好的；如果真要在 **GPU 之间传输数据**，可能需要更好的协议。
- 网络只管高效传 **datagram**；但传输时还有很多细节，例如 TCP 有**拥塞控制**这种难题，还有**断连/连接管理**（连接何时决定中断），这些都是非常重要的、架构里需要包含的设计。
- 其他基本上全部交给别人。
- 讲者总结：在这个基础上，它做好 connectivity 这一件事。

Internet 逻辑链：只做 networking/connectivity → TCP 让可靠传输不必由上层担心 → 上层应用不预设 → 应用在 connectivity 上爆发 → 架构以“变化本身”为永存原则 → 拥塞控制/连接管理等细节纳入架构，其他交给别人。

第 18 部分





架构里先抓住 connectivity：做好连接层，上层世界会变

- 讲者认为，这些是架构里需要包含的设计；**connectivity** 是其中关键的一件，其他东西基本上可以交给别人。
- 讲者的因果链：把 connectivity 做好 → 上面的世界其实变了很多。**（整理者串联）** 下面 Telnet、FTP、HTTPS 的变迁，就是他用来展示这个“上层变化”的例子。

Telnet、HTTP 裸奔到 SSH/HTTPS：安全要求把旧协议挤出去

- 早期还没有“全网 HTTPS”。讲者读书时，大家都在 **HTTP** 上“裸奔”。
- 那时直接用 **Telnet** 连到南京大学 **小百合 BBS** 灌水。
- 今天 Telnet 的安全性等已经达不到要求，所以有了 **SSH**。
- 逻辑：旧协议能用，但安全不足 → 新安全要求出现 → Telnet/HTTP 裸奔被 SSH/HTTPS 取代。

FTP 的断点续传：协议内置能力，后来可被 HTTPS 接口吸收

- 讲者那时传文件用 **FTP**。FTP 好在协议本身就支持**断点续传**。
- 当时在校内 FTP 下电影很开心：网络可能不稳定，或下到一半要停；下次能从断点继续传输。
- 今天 FTP 没有安全等也显得不合适，虽然可能有安全扩展。
- 但今天做软件变容易了，所以可以把这些功能全部实现在 **HTTPS 的一个接口**里。
- 链：FTP 协议内置断点续传 → 安全需求 + 软件开发变容易 → 功能上移到 HTTPS 接口。**（整理者串联）** 这说明能力不一定留在原协议，而可以被更高层通用接口吸收。

HTTPS 像 TCP/IP 上一层：微服务出本地就是 HTTPS，RESTful API 是建议规范

- 讲者说，HTTPS 又变成有点像 **TCP/IP 的上一层**。
- 今天访问所有 **microservices**，只要出了本地、到 Internet 上另一个服务商，全都是 HTTPS。
- 然后有 **RESTful API** 这样的规范；讲者强调这是**建议**，可以不做，也可以不严格遵循。
- 在这个体系上又长出很多东西：在共同规则之上持续**更换局部**。
- 类比：像建设了一个**城市的交通网络**，建筑一直不断向前演进。
- 链：HTTPS 成为通用接口层 → RESTful API 成为共同约定（非强制）→ 共同规则稳定、局部持续替换 → 生态演进。

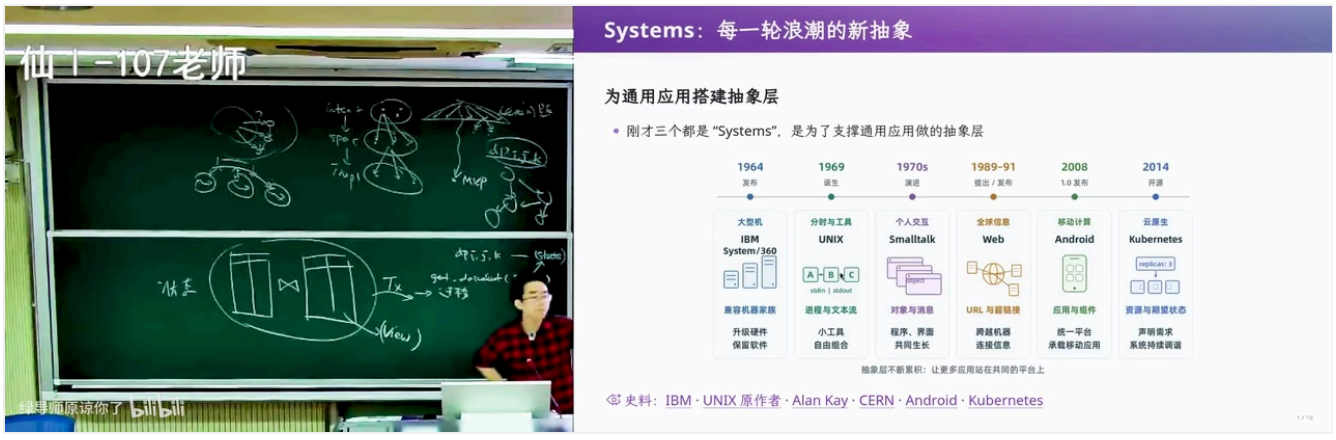
大学课程：具体技术要会，但经典设计如何应对变化更值得看

- 讲者说，在大学里学这些课程，肯定要学具体技术：有一天真的会写 SQL；爬网/拖网时可能还需要一个 SEO（原文如此，含义不确定），得自己手写、有概念才写得出来。
- 但另一方面，更重要的是看到沉淀下来的**经典**：它们为什么能成为经典。
- 讲者讲的是偏**软件架构**的方面：这些系统如何应对变化。
- 可以体会：为什么一个好的设计可以支撑起一个生态，应对这些变化。
- 讲者补充：还是要讲**软件工程**，这是**生成式软件工程**；也还是要讲**软件架构**。

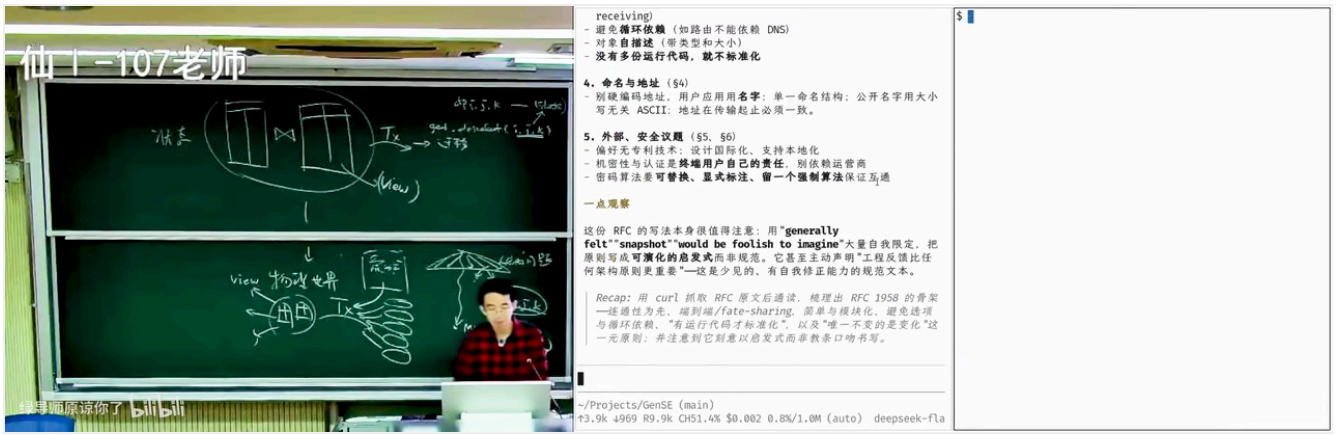
system 是支撑应用的通用抽象层：抽象做得好，才能长期演化存活

- 讲者从刚才的 systems 看到：**system** 是为了提供支撑应用的**通用抽象层**。
- 因此它特别抽象；一般要做得特别好，才能在很长的演化里存活。
- 经典系统谱系（讲者列举）：**IBM System/360**（原文“IBM360360”）→ **UNIX** → **面向对象编程语言** → Internet、Android → 云原生里的 **Kubernetes（原文“Cop cop nis”）**。
- 讲者说 Internet、Android 等应该会专门有一次课来讲。
- 最后提出问题：那我们有没有**面向客户需求的软件架构**呢?

第 19 部分







系统层地基已搭好：OS、数据库、Internet 之上再谈客户需求架构

- 讲者预告：Internet、Android、云原生里的 [Cop cop nis]（原文，具体名词待考）应该会专门用一次课讲。
- 先看系统层：操作系统打好地基，操作系统上面有数据库、有 Internet。这个舞台搭好后，教务系统、图书管理系统等各类业务逻辑系统就都能上线。
- 于是问题变成：在这个抽象层基础上，面向客户需求的软件架构是什么?

1998/2000 的直觉：把每个页面/功能做成一个数据库事务

- 如果回到 1998 或 2000 年，为学校或企业设计软件，手上工具是 Internet、数据库、操作系统。
- 最直接的想法：为每一个页面、每一个功能做一个数据库 transaction。
- 讲者给出的理由：数据库已经把物理世界和过程投影到信息世界，分解成很多表；状态变化由数据库事务驱动。
- 逻辑链：
    1. 物理世界/过程 → 数据库表（投影）
    2. 需求 → 分解成不同需求点

3. 每个需求点 → 一个数据库 transaction
4. 当前状态 → 数据库视图；视图本质也是查询
5. 查询得到切片 → 显示在某个界面位置
6. 由此得到最经典的应用程序架构

- 例子：想看“有哪些学生选了我的生成式软件工程课程”，就写一个 `select`，把数据库视图作为切片取出来。

软件工程书的需求分解，可以对上“需求点 → DB 事务/视图”

- 讲者认为：理解上述映射后，就能明白那些软件工程书为什么这样写。
- 按他的说法，构建业务系统要做的，就是把需求投射、分解成不同需求点；每个需求点变成数据库 transaction；再为当前状态提供各种数据库视图。
- （整理者串联）因此，软件工程里的“需求分析/分解”，在经典业务系统中可直接落地为数据库事务与查询设计。

这个经典架构在“需求点 = DB 事务/查询”时非常好

- 讲者判断：当每个需求点都是数据库事务/查询时，这个架构其实非常好。
- 它催生一波应用浪潮，因素包括：
  - 操作系统；
  - CRUD 数据库；
  - Internet；
  - 1998 年 Java 如日中天时，[WSA98] keynote 中讲到的 Java 语言设计。
- Java 允许直接在 HTML 里写代码。
- 讲者补充：这不是很新的想法；今天这仍是很好的架构，叫模板语言。

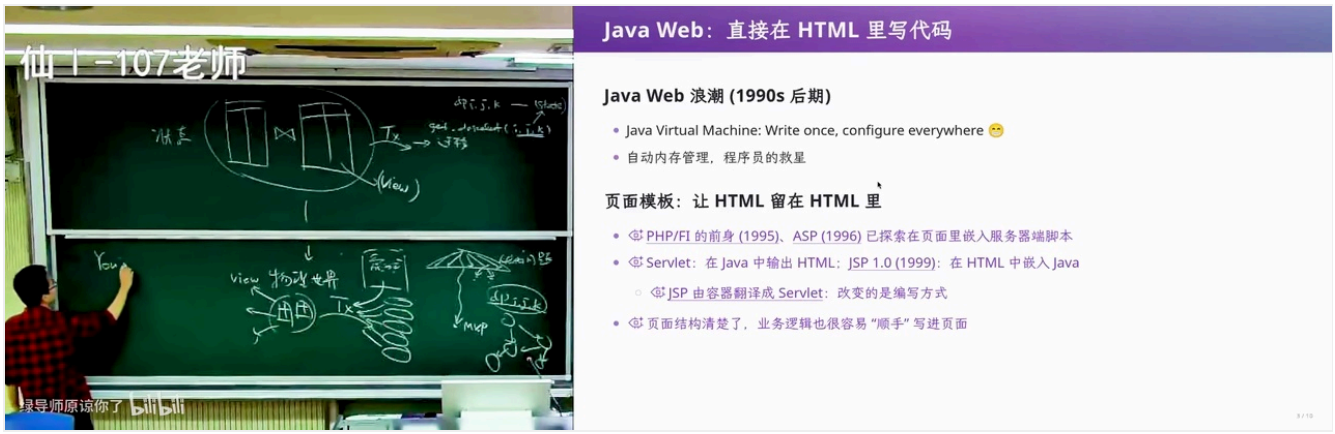
Prompt 拼接的旧问题：A+B+C 不如模板语言

- 讲者问：你们写过 AI 应用、需要定制 prompt 吗？怎么拼 prompt?
- 常见做法是：$[\text{prompt} = A + B + C + \cdots + D]$ 写好多好多 prompt。
- 讲者主张：更好的办法是用一个模板语言。
- （整理者串联）这里与前面 HTML 嵌 Java 的类比一致：把可变内容硬拼进字符串，不如用模板机制组织。

小结：讲者用“需求 → 数据库事务/视图”解释经典业务架构，并把同一逻辑延伸到 prompt：不要手拼 A+B+C，而应使用模板语言。

第 20 部分





用模板语言写 prompt，比在 Python 里拼接更可维护

- 讲者认为，如果要写很多 prompt，再往里加很多变量/内容，一个好得多的办法是用**模板语言**。
- 例子：把 prompt 写成 `You are ...`，其中 `instructions` 是一个变量。模板语言是**可扩展的语言**，有渲染语义：渲染模板时传入 `instructions`，这个字符串就直接贴到对应位置。
- 逻辑链：
    1. prompt 里需要插入多个可变部分（如 instructions、今天的日期）。
    2. 若用 Python 表达式/手工拼接，会很难维护。
    3. 用模板语言声明变量并在渲染时传入，prompt 就变得好维护。
- 讲者举例：在 Python 环境里用模板（原词听不清，疑似 Jinja/Jinja2；后文“金家兔”也对应 Jinja），可以把很多内容，比如今天的日期，传进去。

嵌入服务端脚本的页面模板，是同一类功能的早期实践

- 讲者说，这个 feature 最早在 **PHP/ASP** 时代就已经在探索：怎样在页面里面嵌入服务端脚本。
- 到了 **Java 时代**，有几种不同写法。早期写法很像把 Java 代码和 HTML 模板嵌在一起，甚至能一起循环。今天看 Jinja，里面也有类似功能。
- 有了这种能力之后，开发**以数据库为中心的应用**变得前所未有地容易。

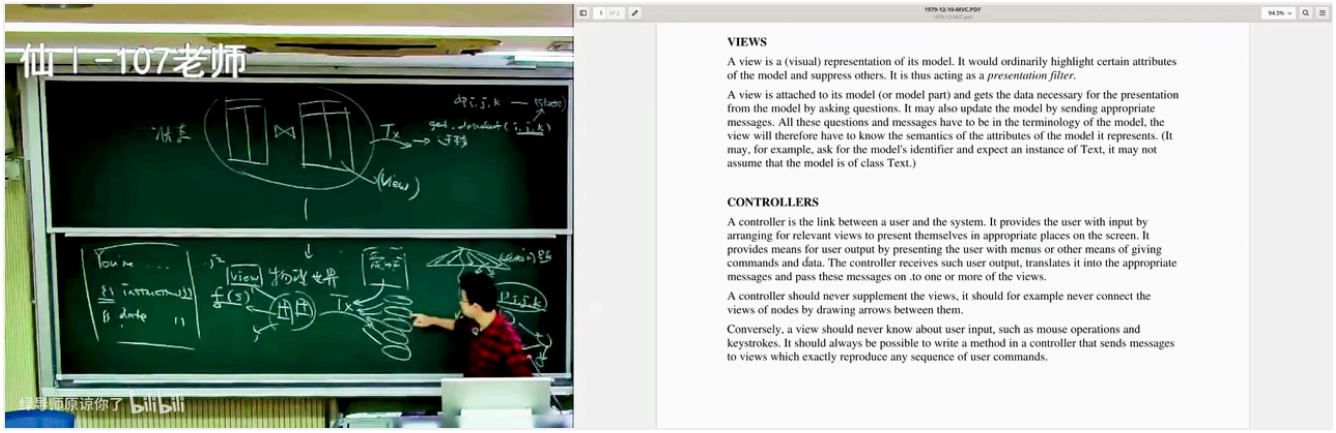
每个页面 = 一个 SQL transaction，然后把数据投射成 HTML

- 讲者描述这种开发方式：对每一个数据库应用，尤其是一个页面，只需要一个 **database transaction**，把所有需要的数据捞出来，放进一些 **list**、**map** 里。
- 然后把这些 map 用模板方式投射成一个 HTML 页面。
- HTML 还有一个很好的设计：把 DOM 数据和样式分开，也就是 **CSS 和 HTML 分开**。样式可以不管，但通过给不同 element 不同样式，可以排出几乎任何形式的界面。
- 因此，一个信息系统页面可以在这里放 button，在那里放任何由代码生成的动态东西。
- 讲者由此解释：像**图书管理系统**这样的系统，为什么在那个时期集中式地爆发了。
- 核心模式是：**每个页面都是一个 SQL transaction**。

从“页面即事务”导出 MVC：Model–View–Controller

- 讲者说，这个想法导出了一个非常重要的模式：**MVC**。他提到一篇很小的、只有两页纸的论文，年份从 1972 改正为 **1979**。
- 逻辑链：
  - **Model**：represent knowledge，是当前世界知识的 **single point**。例如：谁选了什么课、有哪些订单。
  - **View**：是 model 的可视化表示。讲者说，view 是 model 的一个 function，而且应是一个 **pure function**。所以你不希望 view 有任何改变世界的能力。
  - **Controller**：如果你希望有一个操作可以改变世界，就需要 controller。讲者引述：controller 是用户与系统之间的 link，负责用户输入、输出，承载给出命令和数据的意义。
- 讲者的总结：controller 把需求中“能修改的点”翻译成 **transaction**，把你看到的东西变成状态的投射。这是一个非常干净的架构。
- 因此，它很自然地成为任何信息系统的一个**标准架构**。

第 21 部分





MVC 的干净分工：Model、View、Controller

讲者用一句话概括：把操作翻译成 transaction，再把看到的东西变成状态的投射。这个架构很干净、自然，会成为任何信息系统的标准架构。

- **Model**：业务数据。以教务系统为例，是课程、选课记录等。
- **View**：用户看到的页面。
- **Controller**：按钮、输入框等交互；点击后会进入数据库事务。 讲者认为这是大家熟悉的 MVC。

在“整页刷新”前提下，MVC 成立

讲者假设：如果是在 HTML 单页面模式、只有一个 HTML 页面时，每次点按钮后页面都会重新刷新/更新；在这个前提下，MVC 是成立的。

- （整理者串联）因为整页重渲染时，临时界面状态可以随刷新被丢掉，服务器端 Model 足以决定新的 View。

现代 UI 的问题：大量状态不在 Model 里

现代界面上有大量元素的状态不在 Model 里：

- 可随时展开的菜单。
- 搜索框。
- 鼠标 **hover** 到元素时，元素应亮起。

讲者指出：这些也需要被管理，但不应存在数据库里。

搜索框例子：过滤被误做成数据库事务

- View 是学生列表，列表上有搜索框，用户按姓名或学号查找。
- 讲者认为教务系统讨厌的地方：搜索被做成某种 **database transaction**——先在搜索框输入，点搜索，再返回结果。
- 但有时列表已经返回，用户只需要一个**过滤器**。
- 这个过滤器不改变 Model 状态，也不会再查 Model；数据已经在 View 里。
- 但仍然需要改变 View 的状态。 → 所以 View 自己也需要状态管理。

hover 等临时状态也需要 Model，但不能进数据库

- hover 高亮等临时交互状态也需要一个 model。
- 但这个 model 不应该存在数据库里。 → 讲者提出：为什么不把 **View State** 也当成 Model？由此引出 **MVVM**。

讲者称 MVVM 是现在/最近比较时髦的概念。

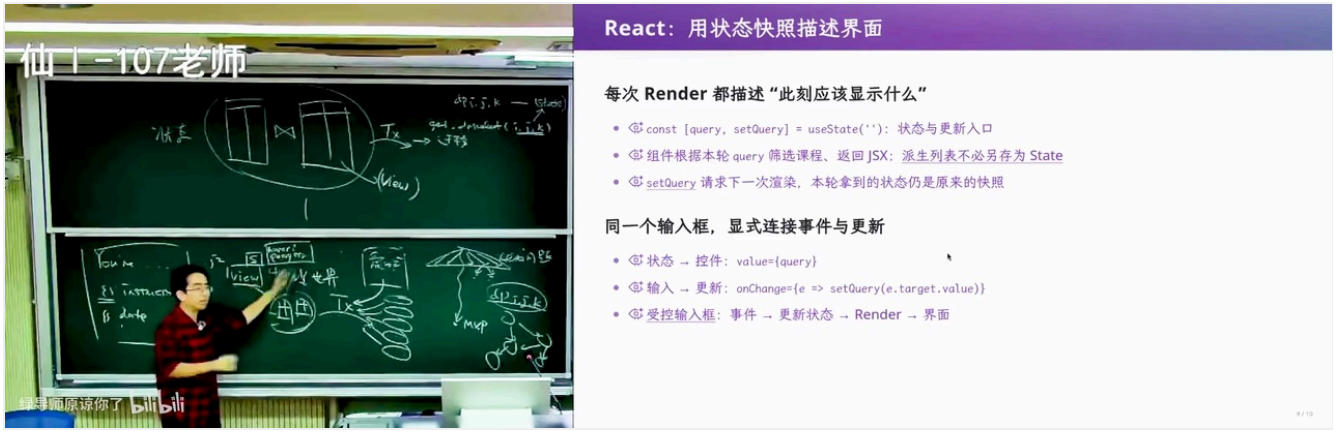
逻辑链：MVC 分工 → 整页刷新下成立 → 现代 UI 出现非数据库 Model 的 View State → 搜索/ hover 证明 View 需要自己的状态 → 把 View State 也当 Model → MVVM。

讲者偏好 React：网页是数据库的投射，View 还有自己的状态

- 讲者说有两个与 MVVM 有关的设计；他不想引起圣战，个人更偏向 **React**。

- 理由 1：React 有 View，对开发者比较直观；写过一点前端就能立即上手。
- 理由 2：React 更纯净、纯粹。
- React 希望保留 Model，也保留 View。
- 在 React 视角下，网页上看到的东西是**整个数据库的一个投射**。
- View 上还可以有自己的状态：是否 hover 了某个东西、**query string** 是什么。
- 这个状态随时可以改。
- 例子：query string 映射到一个文本框，文本框和 query string 的状态绑定；View 里当然也可以绑定这个状态。

第 22 部分



DOM 可以写成状态的纯函数，但性能会立刻惩罚全量重算

- 讲者先举界面状态例子：**query string** 映射到文本框，文本框与 query string 状态绑定；**hover** 也是每个 element 可能有的状态（是否 hover）。
- 于是提出设想：能否把 **DOM tree** 定义为当前所有界面状态的函数：$[\text{DOM tree}=f(\text{当前界面上所有状态})]$ 并且 (f) 是 **pure function（纯函数）**，没有副作用。输入当前界面状态，输出 DOM tree。
- 讲者判断：这肯定可以，理论上成立。
- 但有一个非常严重的问题：**性能**。网页上可能有几万个元素；当 query string 改变——例如输入一个字符——整个状态都要重算。
- （整理者串联）若每次都从全量状态重新计算 DOM，成本会随页面元素规模膨胀，所以不能把“可定义”直接当成“可运行”。

框架用 Virtual DOM / 依赖图隐藏更新复杂性

- 因此框架在系统里做大量 **Virtual DOM** 工作，把这些复杂性藏起来。
- 使用方式变成声明式：
  - 你按它的模型声明**状态是什么**；
  - 声明**状态怎么变、何时变**；
  - 剩下更新不用管，框架帮你更新。
- **Vue（[will/VIVO]）** 更像是帮你组建一个**依赖关系图**：你写 `input = ...` 这类代码时，它建立依赖关系，并帮你保持同步。
- 讲者评价：两者其实差不多；随着 Vue 新版本发布，它们好像变得更像。但讲者说“我们用的比较少，所以我没有发言权”。

共同设计思路：隔离复杂度，把状态和 DOM 翻译分开

- 这两个框架的设计思路都要求**隔离复杂性**：
  - 把**变的东西**与**不变的东西**分离开；
  - 把**逻辑**与**数据源头**分离开；
  - 从而可以分开管理。
- 讲者给出的理由：**“我们是 small head，头很小”**。关注状态时，最好就不要同时关注“状态怎么变成 DOM”；等状态完全确定后，再看怎么把状态翻译成 DOM。
- 两者共同模型：界面由状态决定，界面是一个 **pure function**：$[\text{UI}=f(\text{state})]$
- 但讲者提醒：**更新模型**稍微有一点不一样。
- （整理者串联）因此，这些框架主要解决的是“状态 → 视图”的声明式同步，而不是所有业务复杂度。

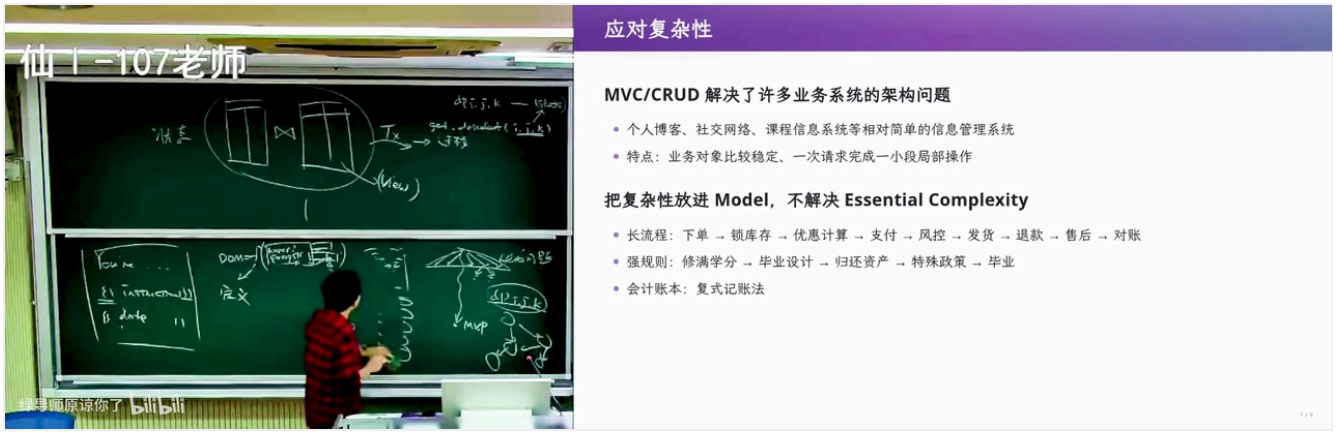
MVC/MVVM 没解决真正的 essential complexity，尤其长流程

- 讲者承认这些可能是做前端开发的人比较熟悉的；但他认为，MVC/MVVM 没有解决真正的 **essential complexity**，尤其是面向**长流程**、真正的长流程任务。
- 例子：假设要实现一个淘宝（讲者说当然不会真去做这么复杂的东西）。淘宝的麻烦不在“下单”本身，而在下单后的流程：
  1. 下单；

2. 库存锁定；
3. 计算优惠；
4. 支付，且支付需要用户确认；
5. 支付后还要：风控、发货、退款、售后、对账。

- 讲者指出，前面用**当前状态**来表示事件，其实做了一个**不切实际的假设**。用当前状态表示事件当然 OK，没问题，可以用**二维表**来表示；但……
- 讲者的追问停在："那我在当前状态下……"
- （整理者串联）长流程跨多个阶段、外部动作和用户确认，并不是单纯把当前界面状态映射成 DOM 就能表达清楚。

第 23 部分



用当前状态表示实体：二维表能画，但"订单走到哪一步"会暴露问题

- 讲者先点出一个**不切实际的假设**：用当前状态表示实体（原文：事件）。他承认，当前状态本身可以用**二维表**表示，没问题。
- 但问题立刻出现：在当前状态下，怎么表示一个订单到底进行到哪一步?
- 最直接的建模方式：
  - 数据库里放一张 **order** 表；
  - order 表里有一列叫 **status**（原文识别为 STATI 等）；
  - 值可以编码步骤：1=下单，2=库存，3=……；
  - 或者把 order 展开成多个字段：是否下单、是否库存、是否支付等。
- 讲者指出：这样会让**状态机变得非常复杂**。

- 复杂点之一：在某个节点触发**风控**后，还要设计它返回的逻辑。讲者认为这是业务系统里的 **essential complexity**，是业务逻辑本身的复杂性。

长流程会制造“不一致状态”：学生既能毕业又不能毕业

- 业务系统越长越大，不管订单管理系统，还是教务系统里的学生，都会出现不一致状态。
- 讲者举例：有的学生处于“**既可以毕业，又不能毕业**”的状态。
- 讲者给出的原因链：
  1. 程序有 bug；
  2. 或者有特批，导致某处被 **override**；
  3. 本来应该触发风控的地方，被另一个人 override 了；
  4. 在错综复杂的逻辑里，系统最后处于**风控失败状态**，但对象已经进入下一环节。
- 于是问题变成：这个数据到底是什么？怎么解读?
- 讲者认为，这类不一致数据甚至会**静默损坏（silently corrupt；原文：silently cpt）**。他相信学校教务系统里有“无穷多”这种数据：经常手工操作、人工 override（原文：手操工程师、手操教学），很多根本不对、不一致的数据留在里面；最后好像没影响同学毕业，也就只能不管。
- 根因在于：业务流程是一整条很长的流程，既要编码过去，又要编码现在，也要编码未来。当前状态本质上只是**现在的快照**，但这个快照是由过去推演过来的；一旦“怎么过来的”丢了，一致性判断就会变复杂。
  - （整理者串联）也就是说，只存当前状态会把导致状态的事件/操作历史丢掉，出现矛盾时难以解释和追责。

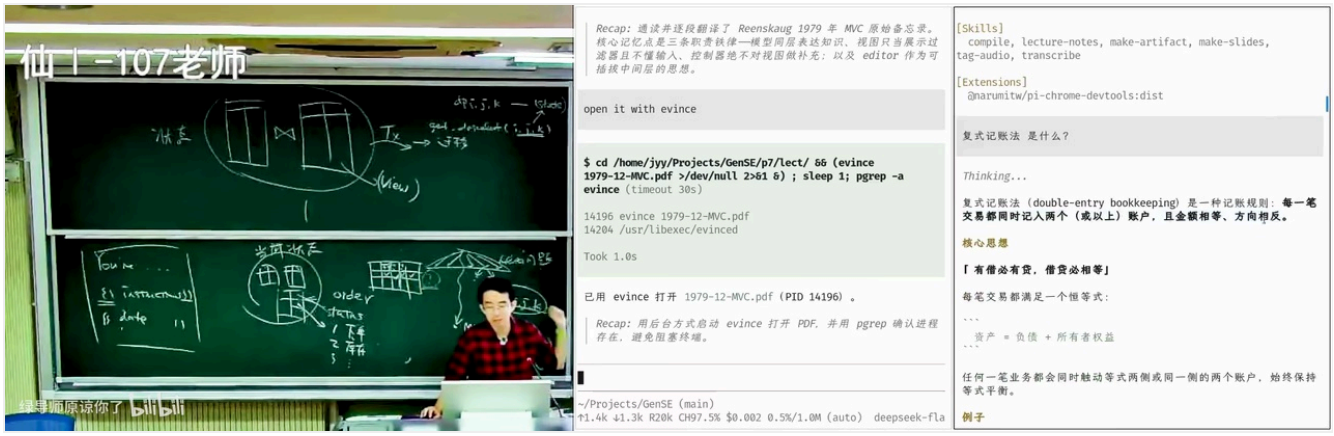
复式记账：用成对、等额、反向的记录获得自平衡校验

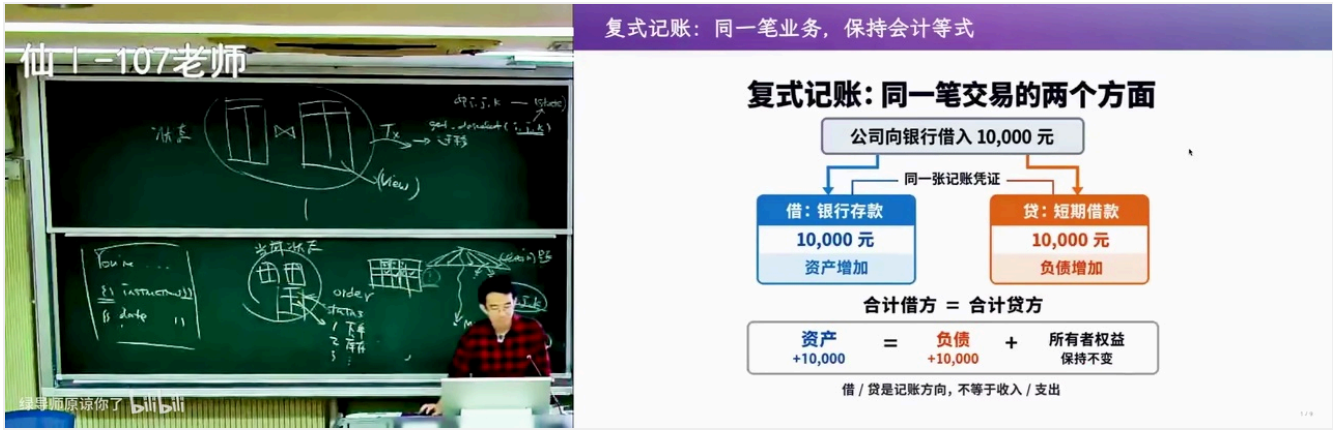
- 讲者换了一个更简单的例子：记账。支付宝会给用户记账；但真正严肃的记账，会计有要求：**复式记账法**。
- 目的：**防错**，防止数据库不一致；一旦发生不一致，能把它检查出来。
- 要求：每一笔交易同时计入**两个或以上账户**，且**金额相等、方向相反**。
- 逻辑链：单笔交易 → 同时记入两个或以上账户 + 金额相等 + 方向相反 → 可**试算平衡** → 不一致可被检查出来。
- 对比单条记录：

    - 只记一条：简单，但无法反映业务的完整结构；
    - 也不具备**自平衡的校验能力**；
    - 复式记账则可以做**试算平衡**。
- （整理者串联）这跟前文的问题是同一类：只留单条当前结果，难以校验；留下成对、可对账的变动痕迹，才能发现不一致。

小结：单条当前状态能表示“现在”，但会丢失“怎么来的”；复式记账用强制成对记录和试算平衡来对抗这种不一致。

第 24 部分





复式记账的价值：单条记录简单，但只有成对记录才能自平衡和追查错误

- 讲者先给出成对记账的基本要求：**金额相等、方向相反**。

- 如果“只记一条”，优点是简单；缺点是：
    1. 无法反映业务的完整结构；
    2. 不具备**自平衡校验能力**。
- 成对记录可以做**试算平衡**。
- 讲者认为，人类发明这种记账方式，是因为**当前状态很可能是 corrupted（损坏/错误）**，是不对的；一旦不对，就需要在一定程度上把“为什么不对”找出来。
- 逻辑链：只记一条 → 简单，但不能反映完整结构，也不能自平衡校验；金额相等、方向相反 → 可试算平衡 → 能校验并追查当前状态是否/为何出错。

需求驱动：不懂会计，也必须产出合规的资产负债表

- 讲者说明自己的需求：要记一张**资产负债表**。
- 这张表大致包含：**现金**、**存货**、**固定资产**、**应付账款**、**短期借款**、**所有者权益**等。
- 如果要按符合会计法规的方式记账，就必须要得到这样一张表。
- 但讲者声称自己对会计没有任何知识。于是问题变成：**在完全不懂会计的情况下，怎样设计一个好的信息系统/软件架构来满足这个记账需求？**

用 event sourcing 绕开会计知识：不记当前状态，改记状态迁移

- 讲者说自己用了 **event sourcing（事件溯源）**（原文 ASR 近似“一般 sourcing / 一般缩 ING”）。
- 核心转变：不再记录数据结构的**当前状态**，而是记录**所有状态的迁移**，也就是记录它是怎么来的。
- 因此系统本质上是一笔**流水账**。例子：今天收了 1000 块钱，明天又减了 1000 块钱，以及为什么。
- 讲者说自己不会记账，也没有设计数据库系统、没有设置数据库；他直接让 AI 设计了一个小的、**像编译器一样的东西**。

AI 生成映射：从资产负债表条目反推每笔业务的多表变化

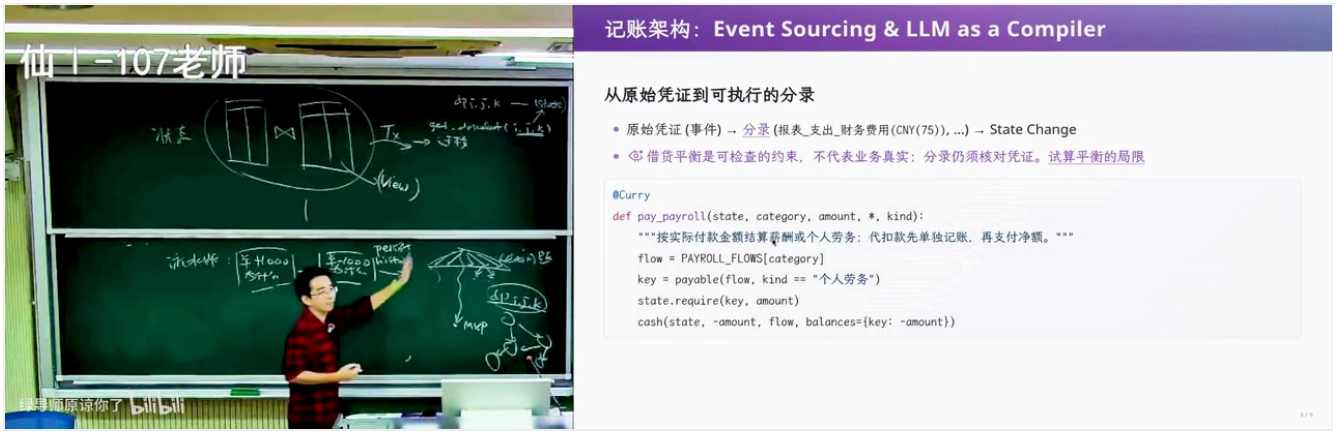
- 讲者有很多“记账的动作”，这些动作是从最终需要交付的**资产负债表的所有可能条目**中提取出来的。
- 例如要付钱、付**营业成本**：每付出一笔，都会在这些表格上记录：

    - **货币资金**会怎样变化；
    - **所有者权益**会怎样变化；
    - **应付账款**会怎样变化；
    - **现金流量表**上会发生什么。
- 讲者指出，这套表格完全是 **AI 生成**的。
- 他的选择是：**选择相信 AI**，以及相信最终校验会找出这里面是否会有什么问题。

架构核心：不可篡改流水账 + LLM 编译器 + 最终校验

- 讲者概括的架构设计是：**用 LLM 做编译器**。
- 系统中有一个 **persistent history（持久历史）**，即**不可篡改**的流水：发生了什么，就写什么。
- 例子：如果今天要允许某一个不能毕业的同学毕业，也必须在其中写一条 **persistence（持久化条目）**：允许某某毕业。
- 这样的流水账条目会用 **LLM 编译**成刚才看到的一个个小操作。
- 逻辑链：记账需求 → 不记当前状态而记状态迁移（流水账） → 流水账不可篡改 → LLM 作为编译器把每个条目编译成小操作 → 最终校验检查问题。

第 25 部分



当前状态不是直接维护的，而是由“流水账条目→小操作→重放”得到

- 讲者先交代总体架构：像“我要允许某某某毕业”这样的业务事实，会先形成一条**流水账条目**。
- 这样的条目会用 **LLM** 编译成前面展示过的**很多小操作**。
- 每个小操作都会声明自己的影响：
  - **资产负债表**怎么改；
  - **第二张表**怎么改；
  - **现金流量表**怎么改。
- 当前状态由所有这些操作 **reduce** 得到。逻辑链是：
  - 流水账条目 → LLM 编译成小操作 → 每个操作声明报表副作用 → 按时间顺序 **replay**（转录作 replace）一遍 → 得到当前可提交的资产负债表。
- 讲者说，这是总体架构；里面还有很多小架构。

实现 trick：合并 GPT 写的冗长代码，让记账不再成为问题

- 讲者提到，一开始用 **GPT** 写的代码很糟糕，非常非常长。
- 他用了一个小 trick 把代码合并起来。
- 例如给 **PAYRO**（[payroll?]）加一个参数，就能得到一个 payroll，相当于参数 `state` 已经确定。
- 剩下的函数也用了编程中非常常见的小 trick。
- 讲者认为好处是：**记账不再是任何问题**。
- 示例：银行收支隔一段时间要整理；这些实际发生的记账条目在报表上的所有 **side effect** 都会体现在这里；把它们按时间顺序推一遍，就得到当前报表。
- 讲者把这种模式称为 **event source**，按上下文即 **event sourcing**。

用 database 管“现在”的代价：状态可变，逻辑一变就从不一致到错

- 讲者从这里引出对比：如果设计信息系统时用 **database** 管理信息，database 很自然管的是“现在”：
  - 学校现在状态是什么；
  - 每个学生是什么；

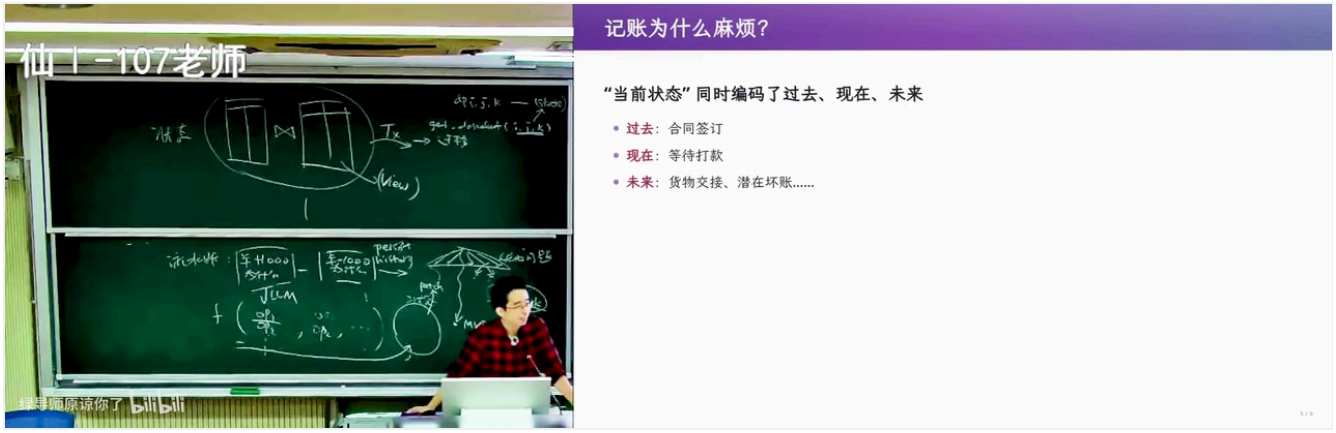
    - 学校有哪些课程；
    - 每个学期有哪些课程。
- 它代表一个**当前事实**。
- 但当前状态是**可变的**。逻辑链：
    1. 状态可变；
    2. 一旦逻辑有错，或逻辑改变，状态很容易造成不一致；
    3. 你会从一个对的状态走到一个错的状态。
- 讲者重申：**好的架构是能够生长的架构**。它能容纳突然多出的新需求，也能容纳过去某个规则被改掉。

规则改掉就要给状态打 patch，而数据库迁移非常危险

- 当过去的规则改掉时，讲者说你需要为状态生成一个 **patch/补丁**。
- 状态的补丁、**数据库的迁移**，在讲者看来是非常危险的操作。
- 他举例如：培养方案改错了、修正了、成绩更正了。
- 其中**成绩更正**会带来大量后果，比如成绩从及格变成不及格。讲者的故事：
    - 假设一位同学想找老师要分；
    - 老师劝他不要要了；
    - 同学非要查卷子；
    - 查卷后，老师发现当时已经救了他一手：最后那个题本来空着，还是给了 20 分；
    - 这下查卷了，只能把这 20 分扣掉；
    - 于是这位同学挂了。
- 讲者说，这对数据库就是一次修改，而且可能是**不在原有业务逻辑里面的修改**。
- 这种修改对数据库是摧毁性的，因为这门课从挂了以后……

小结：讲者用记账流水线说明 event sourcing 的重放得到当前状态；又用成绩更正说明直接改当前状态、为状态打补丁/做迁移会引发连锁后果，因而危险。

第 26 部分



在当前状态数据库里做“非业务逻辑修改”，会破坏内部一致性

讲者先设定一个数据库场景：如果做一个不在**原有业务逻辑**内的修改，对数据库可以是摧毁性的。原因不是改一个字段本身，而是字段之间有依赖。

讲者给的逻辑链： 课程挂科 → 学分数减一 → 影响能否毕业、能否准出、能否选下一门课 → 当前状态数据库内部已经不一致。

他举**操作系统先修课**的例子：
要选操作系统，先修计算机系统基础；某学生因为计算机系统基础通过，所以选上了操作系统。后来计算机系统基础挂科了，他还能选操作系统吗？讲者说，这时数据库“内部天生就是不一致的”——先修通过这一事实已经反转，但操作系统选课状态还按旧事实存在。

讲者认为，**essential complexity** 里大量存在这种依赖关系：“我依赖你，你依赖我”。

只把“挂科”从 1 改成 0 不够：你不知道还该改什么

在当前状态 database 中，如果把“课程成绩/是否挂科”从 1 改成 0，看起来只改了一个字段。但讲者强调：你对数据库剩下的部分到底做了什么影响，很难评估，很容易漏掉。

逻辑链（讲者陈述）：

1. 只维护当前状态 → 修改一个事实后，不知道哪些派生状态依赖它。
2. 依赖藏在当前状态里 → 很难评估、容易漏。

3. 所以要用 **Event Sourcing（事件溯源）**：不维护当前状态，而是搞清楚所有原始数据之间的 dependency。

Event Sourcing 的关键：保存原始数据之间的依赖，而不是当前状态

讲者说，Event Sourcing 的思路是“不要维护一个当前状态”，而是把所有原始数据之间的 dependency 搞清楚。

他举了一个 [LM 编译器] 式/记账式的例子：从一个记账条目到一个操作，也可能出错；但出错后可以人工审核，把错的改回来，且“不会付出任何代价”——比如翻译错就直接改正。

更关键的是选课例子：如果这个同学通过了 ICS 课，很多结论是后来算出来的；当“通过”反转成“不通过”后，因为不维护当前状态、保留所有计算，就可以把所有计算重新做一遍。哪怕只用**自动数据流分析**，也能知道“通过 → 不通过”会影响哪些结论。

讲者的原因是：**data flow 被 make explicit**，不再以状态形式藏在当前状态里。他也提到，如果发生“超额选课”这类选课影响其他课程的情况，在这种架构里你会知道改了事实后会影响什么。

（整理者串联：因此这里的优势不是“自动改对”，而是依赖显式化后，可以从原始事实重新派生所有受影响的结论。）

Replay-based：当前状态由过去决定，所以能推演未来状态

讲者认为 Event Sourcing 还有很多“非常好玩”的特点。它是 **replay based**：当前状态完全由过去决定，所以可以用它推演未来的状态。

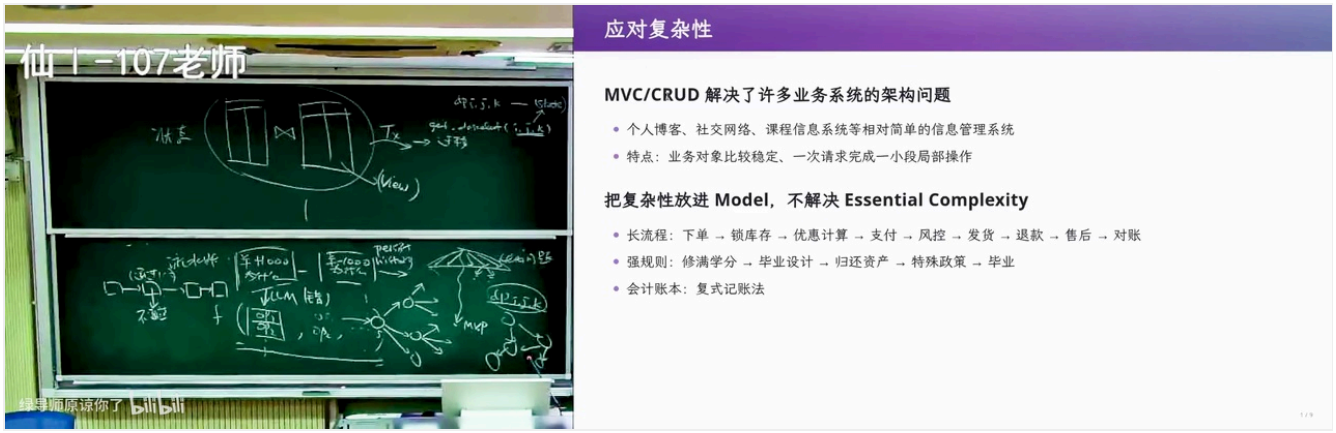
他举例一个流程：**下单、锁库存、优惠计算**；假设现在停在“支付”这一步。因为这个架构天生支持推演，所以可以在这个位置直接推演：

- 如果风控成功，状态会到什么样；
- 如果风控失败，状态会怎么样。

讲者进一步说，如果有一个账本，就可以在 Event Sourcing 的基础上评估“我到底会不会破产”“我破产的概率有多少”。他最后说到：对方只要有一个模型，我只要知道……（此处转录未展开）。

小结： 讲者的主线是：状态式数据库把依赖藏起来，局部修改会留下不一致；Event Sourcing 显式保存依赖，使事实反转后可重放计算，并可用过去推演未来。

第 27 部分



Event sourcing 能把账本变成对未来破产概率的评估器

- 讲者认为：如果有一个账本，就可以在 **event sourcing** 基础上评估自己会不会破产、破产概率多大。
- 评估链：有账本事件记录 → 有一个模型；知道比如应收账款中有多少收不回来就会破产 → 就能对未来做评估。
- 讲者评价：这是“一个好的架构”所能带来的能力。

Event sourcing 不消除 essential complexity，只把事实与预期分开

- 它不能消除 **essential complexity**：先修课依赖、毕业依赖等，都是已经存在的事实。
- 但 event sourcing 把 **事实** 与 **预期** 分开。世界如果没按预期发展，不是让工程师手改数据库、把数据库拖进越来越糟的状态，而是**追加新的事实**。
- 毕业例子：追加“领导认为这个同学可以毕业，其他都不管”这个事实；因为有这个 ORI（转写词，具体机制讲者未展开），他就可以毕业。
- 关键效果：数据库不再靠手改来匹配新现实，而是把新现实作为事实追加进去。

可溯源的计算过程 = 显式建模 data dependency

- 讲者用一句话概括：**“现在是过去的结果”**。
- 有一个**可查证、可溯源的计算过程**，就显式建模了系统中各种 **data dependency**；即使没有真用一个数据结构（例如 **data dependency graph**）去声明“这个表的数据依赖那

个表的数据”，计算逻辑本身仍然可溯源。

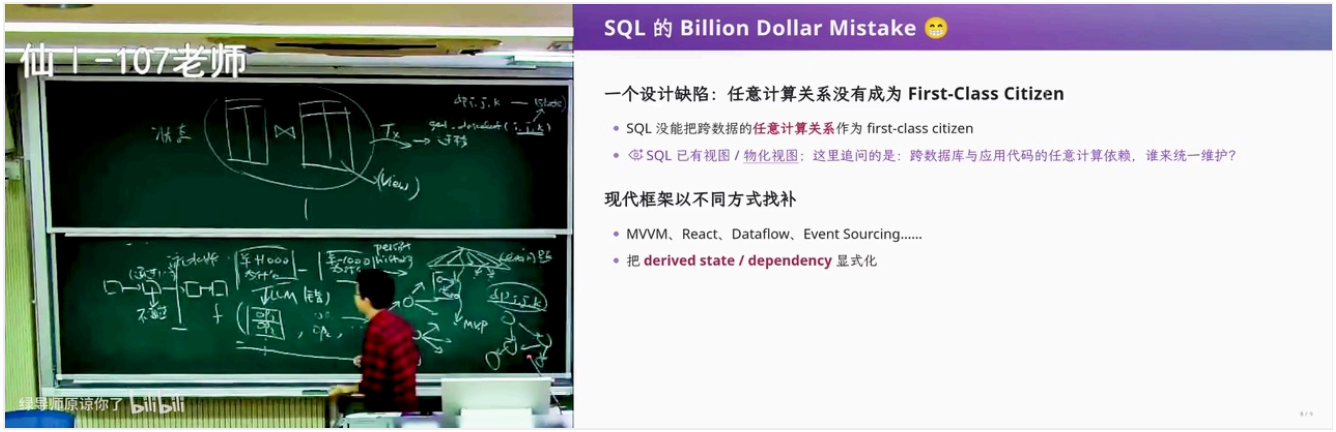
- 例子：因为通过了 **ICS（计算机系统基础；转写中作 ISS/SSS）**，所以要加 ICS 的学分；要从“那个时候”取出该时刻 ICS 课的学分，再加进去。
- 因为课程学分可能变化（例如确实减了学分），有 event source（转写作 source）就能知道当前/当时那个时刻的学分是几，避免“学分还够不够毕业”的问题。

关系数据库的 billion dollar mistake：没有把图灵可计算的计算关系作为 first-class citizen

- 类比：**Tony Hoare** 自嘲发明了空指针，是 **billion dollar mistake**；因为 null pointer 会让程序崩溃，甚至火箭爆炸。
- 讲者认为，在这个意义上 **SQL/关系代数（转写作 sickle）** 也犯了一个 billion dollar mistake。
- 但他强调不能说成单纯设计缺陷：关系代数/关系数据库在当时是**非常可扩展、非常好的架构**。
- 问题在于：为了管理数据，它没有把任何**图灵可计算的计算关系**作为 **first-class citizen** 管起来；这是 relational database 天生做得不好的地方。
- 例子：数据库里放字段，比如学生表里有“能不能毕业”的 0/1 字段；几乎数据库都会这样设计。

小结： 讲者的主线是：event sourcing 不消除依赖，但把事实与预期分离，使计算可溯源；关系数据库则没有把计算关系当作 first-class citizen。

第 28 部分



查询要快，所以数据库常把“能不能毕业”这种判断直接存成 0/1

- 讲者举学生表为例：里面有一个“能不能毕业”字段，值就是 0 或 1。
- 讲者说数据库几乎都会这样设计：**必须快速把它查询出来**。
- 讲者给出的原因是：这类判断背后可能是很复杂的计算，不能用关系代数去建模一个非常复杂的计算；于是把结果落成字段更实际。
- （整理者串联）这意味着这个 0/1 是**物化/派生结果**，不是独立于其他数据和规则的事实。

这个落库的 0/1 成为数据库里的潜在不一致点

- 讲者直接判断：这个时候，这个 **0 和 1** 就成了数据库里面**不一致的潜在的点**。
- 逻辑链可写成：
    1. 为了快查，把“能否毕业”物化成 0/1。
    2. （整理者串联）这个值依赖别的原始数据/规则。
    3. （整理者串联）原始数据/规则变化后，必须同步更新这个 0/1。
    4. （整理者串联）若更新遗漏，字段值就会与真实判断不一致。
- 所以问题不是 0/1 本身，而是它把**派生状态**藏进了一个普通字段里。

现代框架用不同方式“找补”：把派生状态和依赖显式化

- 讲者列举：**MVVM**、**React data flow**、**Event Sourcing**。
- 讲者认为，这些框架都是在用不同方式**找补**。
- 它们共同试图做的是：把 **derived state** 和 **dependency** 显式化（原话作“显示化”）。
- 用链条表达： 旧设计把派生结果物化并留下不一致风险 → 新框架显式表达“状态从哪来、依赖什么” → 用不同机制减少/管理这种不一致。

回头看旧设计，是为了理解新设计为何这样做

- 讲者建议：去看一看那些“老旧的东西”，再理解新设计。
- 他举例问：为什么要做 **Event Sourcing**?
- 讲者认为，把旧设计和新设计连起来看会比较有意思。